GNN Acceleration: Full-graph based learning

2026 MINDS Lab Meeting

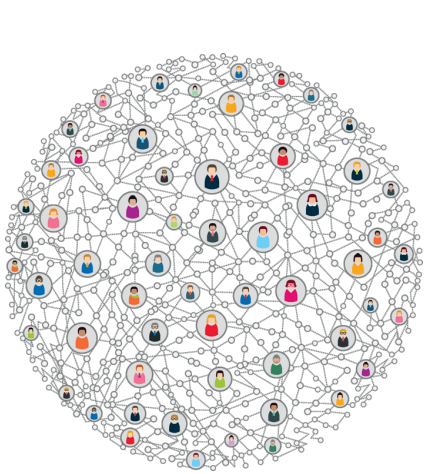
Master's Program Student at Chung-Ang University

SaeJoon Park

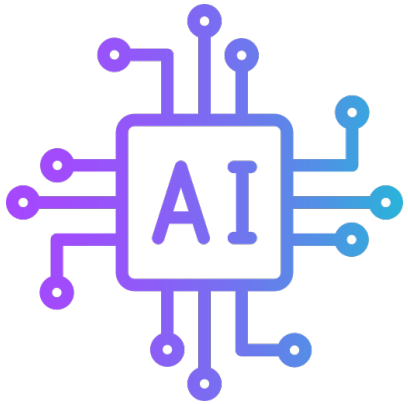
Contents

Background-Graph

- **What are Graphs?**
  - Graphs model entities (nodes) and their connections (edges).
  - Even time series can be graphed by defining who influences whom.
  - Computers encode these relations as numbers (tables/matrices) for learning

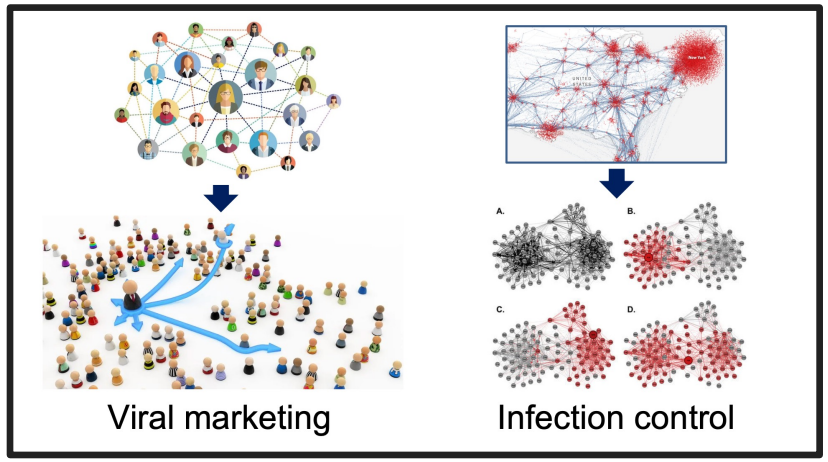


$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

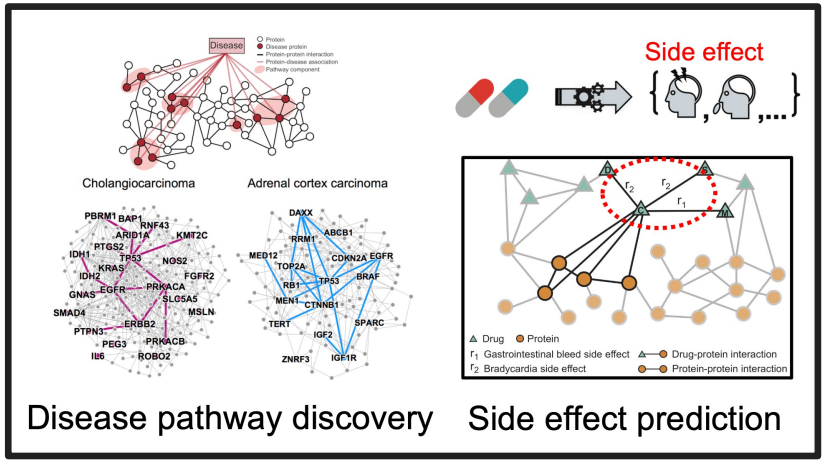


Various Applications

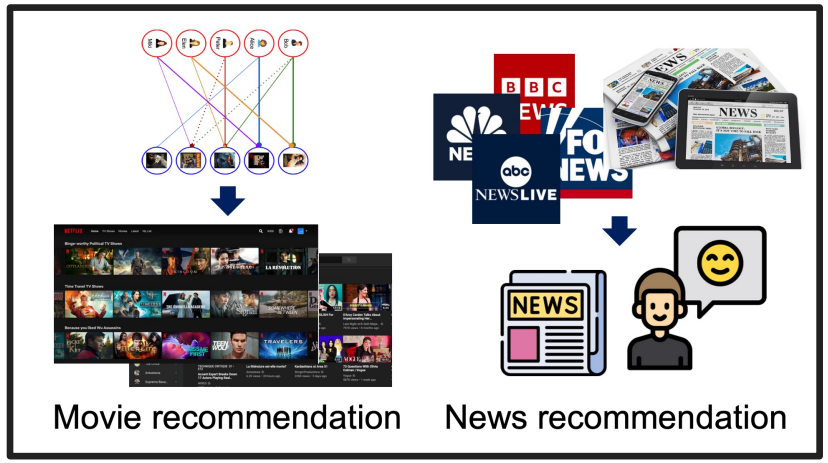
Background-GNN Applications



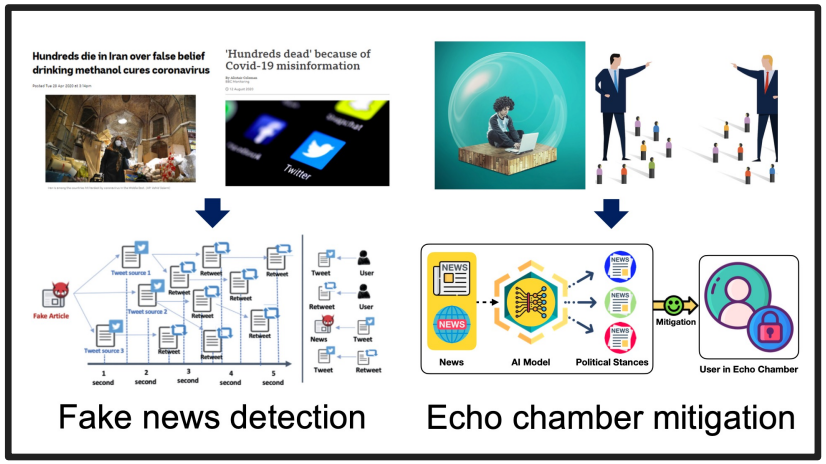
Social network service



Digital Healthcare



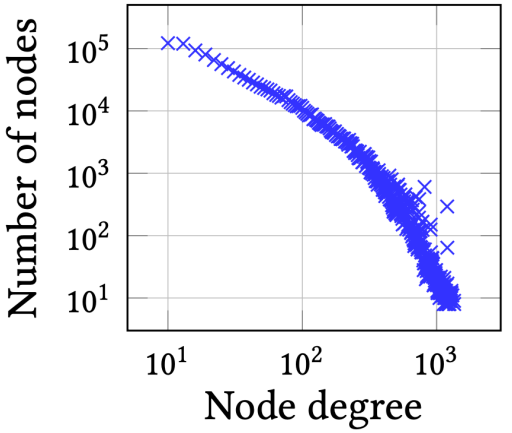
Recommender System



Social Welfare

Background-Matrix

- **Dense Matrix**
  - A dense matrix has non-zeros almost everywhere
  - Graphs are sparse, but dense storage keeps even the zeros
  - Memory waste



$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Too many zeros…
Memory waste

- **Sparse Matrix**
  - A sparse matrix is mostly zeros
  - The key is storing only non-zeros (e.g., CSR) not the zeros
  - It's efficient for non-zero-only ops like SpMV/SpMM
  - This is the common format in GNNs, often as CSR

Background-Compressed Sparse Format

- **Why should we use the CSR format?**
  - Saves memory by not storing zeros.
  - Enables fast access to node i’s neighbors.
  - Optimized for SpMV/SpMM performance.

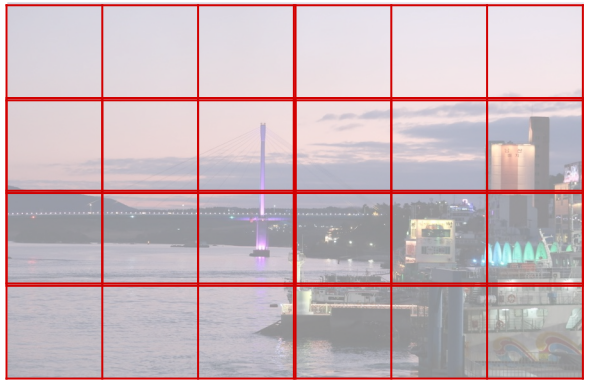
CSR Format

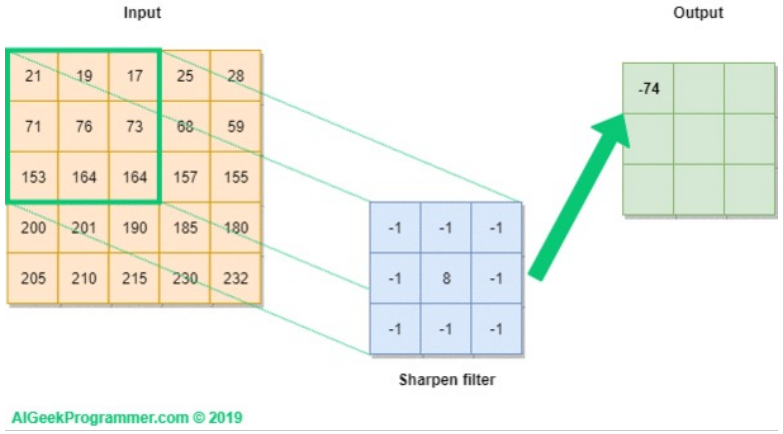
$$A_{IJ} = \begin{bmatrix} 10 & 0 & 0 & 12 & 0 \\ 0 & 0 & 11 & 0 & 13 \\ 0 & 16 & 0 & 0 & 0 \\ 0 & 0 & 11 & 0 & 13 \end{bmatrix}$$

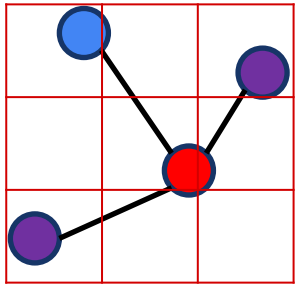
values	[10, 12, 11, 13, 16, 11, 13]	Non-zero 값들을 row 순서대로 저장
col_index	[0, 3, 2, 4, 1, 2, 4]	각 값이 속한 column 인덱스
row_ptr	[0, 2, 4, 5, 7]	각 행(row)의 시작 index (누적합)

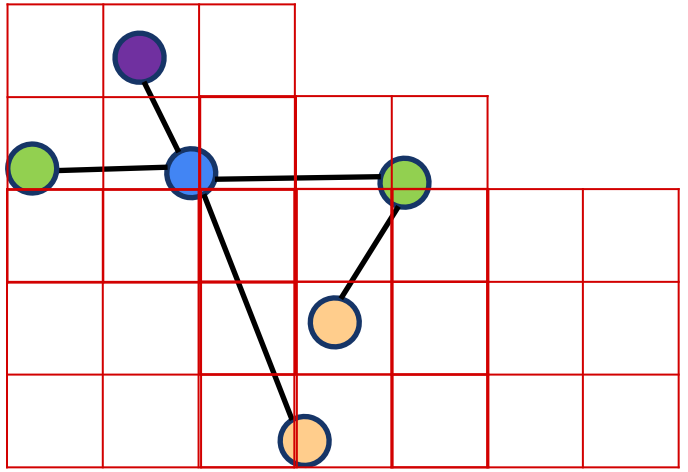
Background-Traditional Way

- **Why are CNNs and DNNs not well-suited for graph data?**
  - Data structure mismatch
  - Each node has a different neighbor structure, a fixed filter cannot be used
  - This issue arises from the inherent characteristics of graph data









Background-Graph Neural Network

- **What are the key characteristics of GNNs?**
  - Message passing is the core mechanism of GNNs.
  - Stack more layers, a node can incorporate information from more hops

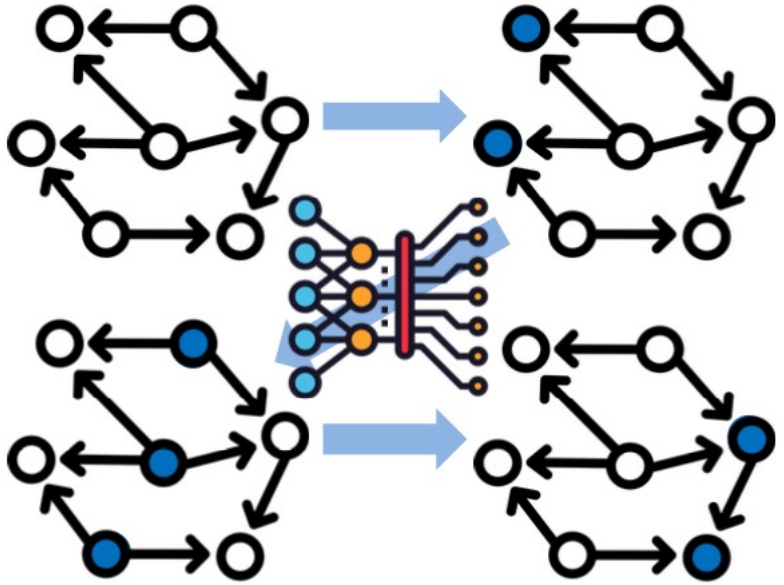
$$AXW + b$$

$$A: adjacency\ matrix$$

$$(AX)_i = \sum_j A_{ij} X_j$$

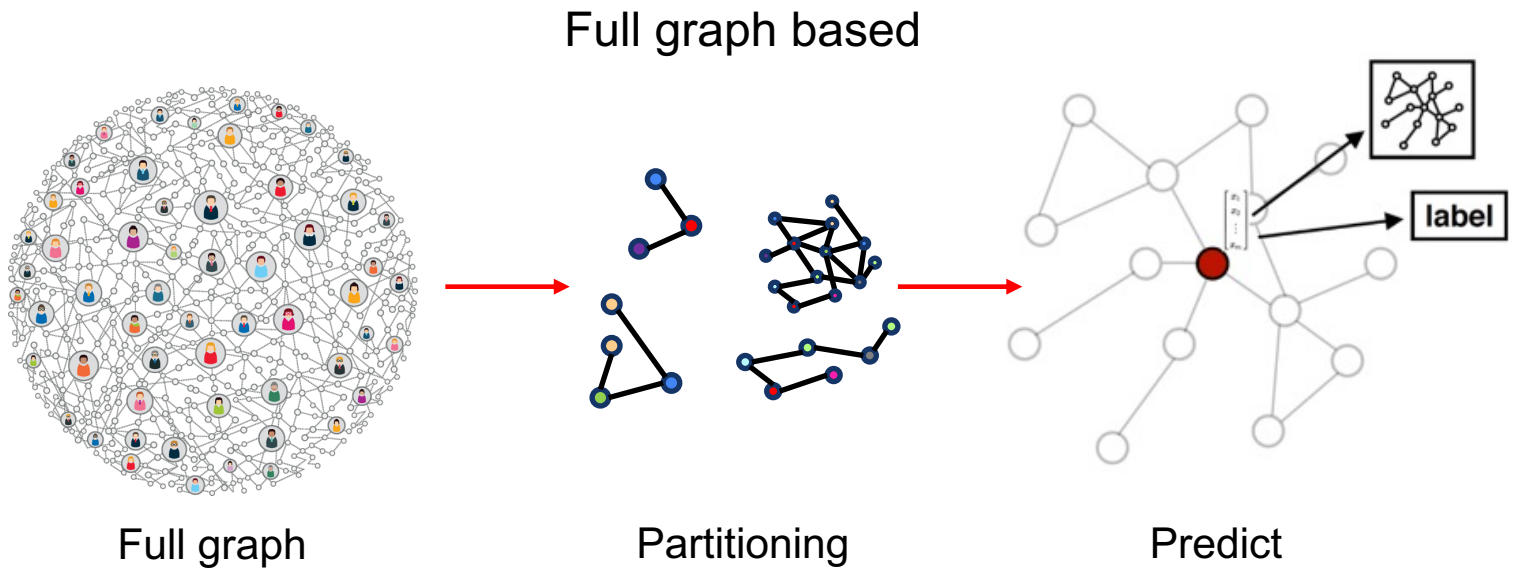
AXW: *Neighbor features are linearly projected with* W

b: $Shift\ parameter$



Background-Full Graph Based Learning

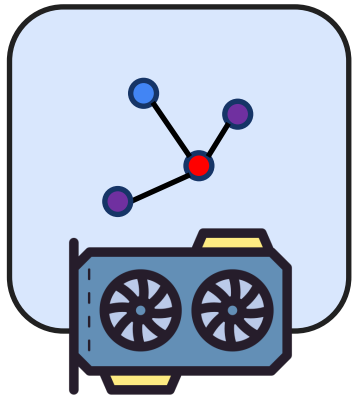
- **What are the key characteristics of GNNs Full graph based learning?**
  - Train on the full graph without dropping any data
  - As the graph grows, memory usage and compute cost rise sharply
  - Deeper layers increase intermediate activations and neighborhood expansion, creating bottlenecks



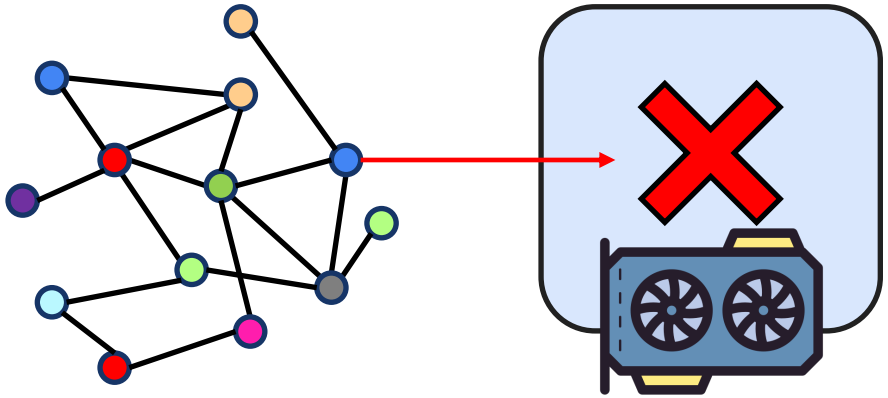
Background-Graph Scales Up Issues

- **What issues arise when the graph scales up in GNNs?**
  - $AXW + b$, the size of A grows as the graph becomes larger
  - A is small enough, it can fit in a single GPU's memory for training
  - A becomes large, it may be hard to handle even in CPU memory, not just GPU memory

$A \downarrow$



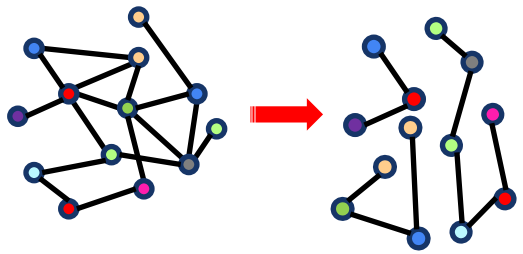
$A \uparrow$



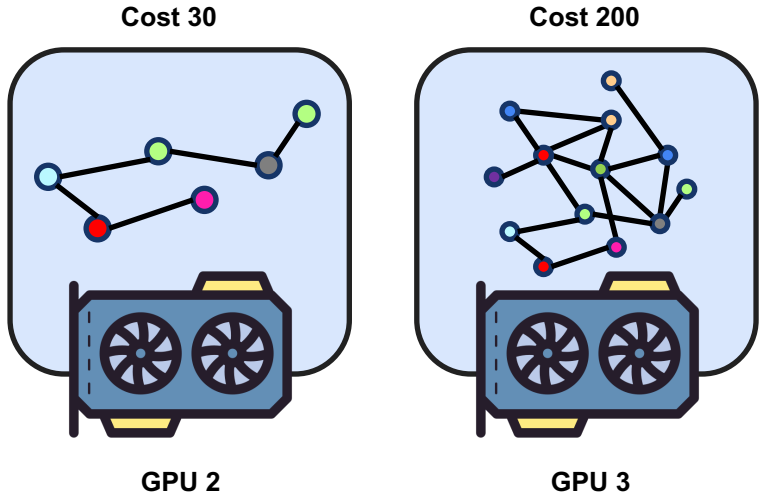
Background-GNN's issues

- **Three technical issues in Full Graph Based Learning**
  - Partitioning
  - Workload Unbalance
  - Memory Manager

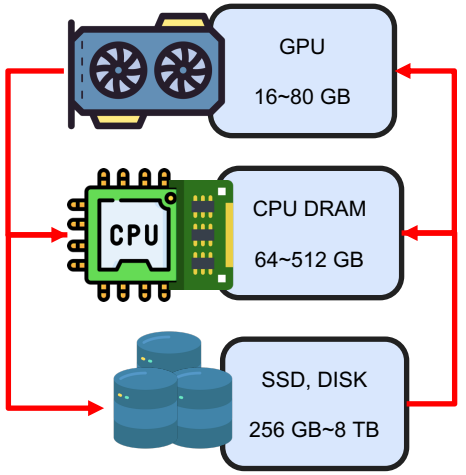
Partitioning



Workload Unbalance

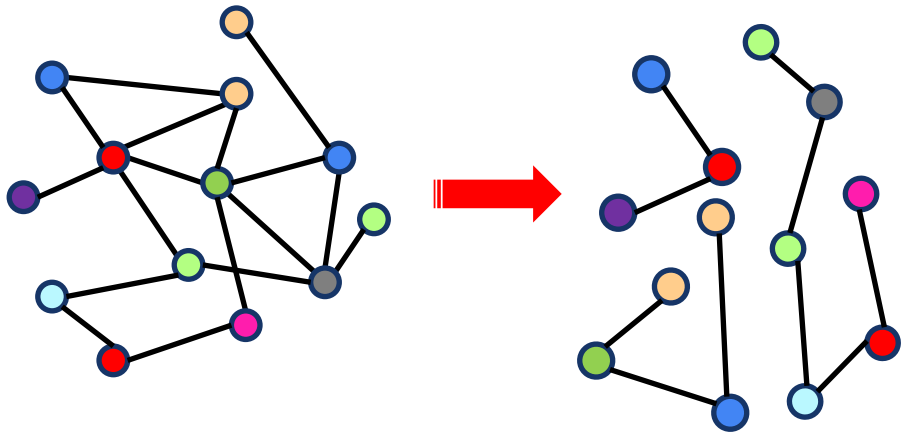


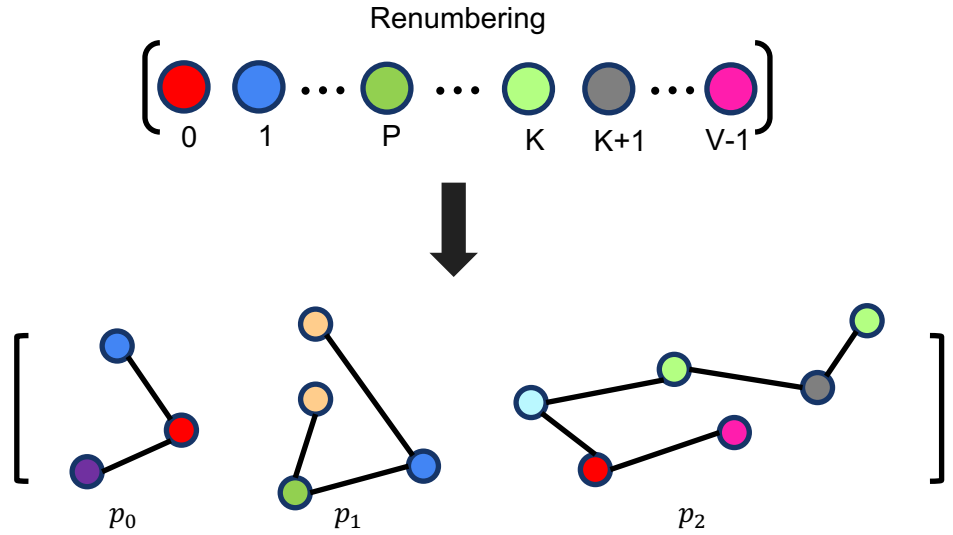
Memory Manager



Background-Partitioning

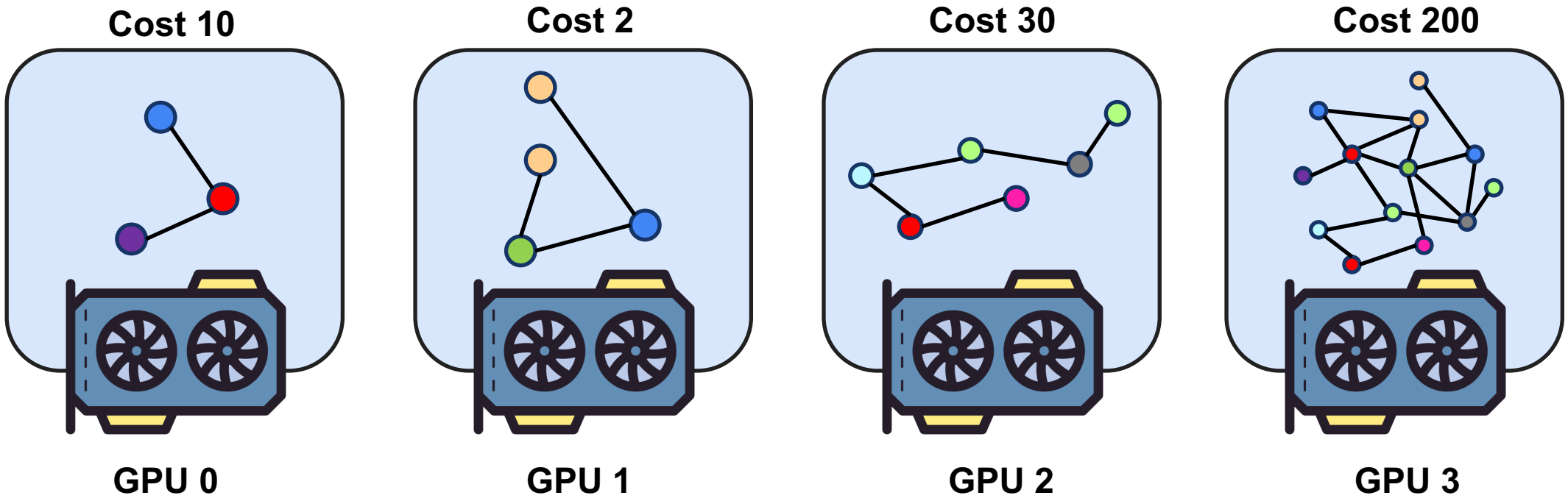
- **What are the benefits of partitioning?**
  - Split a large-scale graph into smaller subgraphs
  - Reduce memory usage and improve feasibility
  - Improve data locality





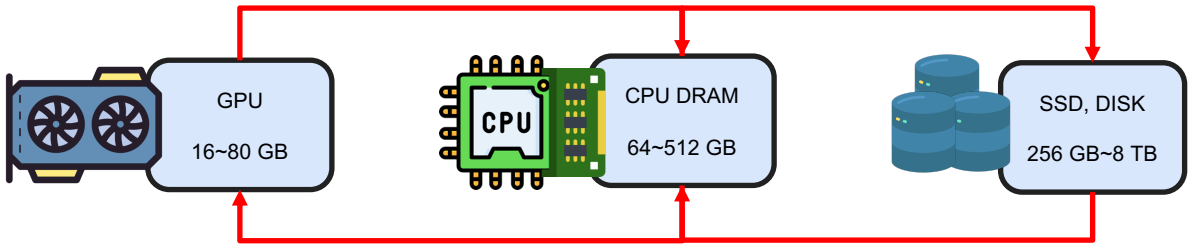
Background-Workload unbalance

- **What problems does workload imbalance cause?**
  - Each subgraph can have a different runtime
  - Some GPUs sit idle while waiting for the next operation
  - Reduces GPU utilization and increases overall runtime



Background-Memory Manage

- **Why do we need memory management?**
  - GPU memory is not large enough and GNN training, intermediate data can grow explosively
  - We need to use CPU, DISK memory
  - Careful memory management essential



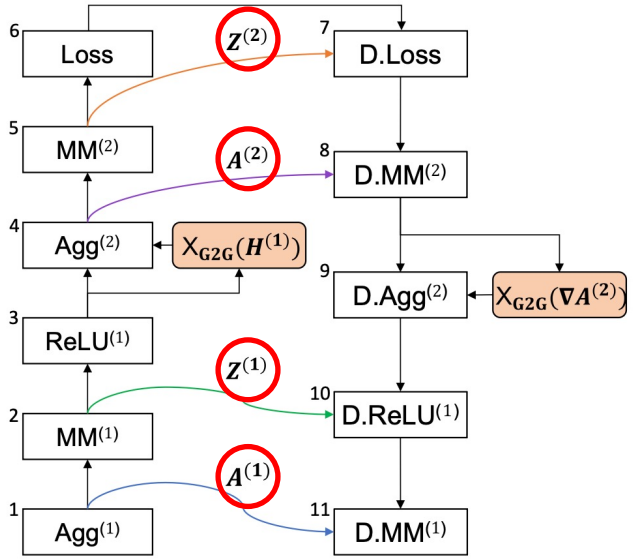


Table 1: Sizes of graph, vertex, and intermediate data when training GNNs on real-world graphs.

Dataset			Feature dimension			Graph data	Vertex data		Intermediate data	
name	$\|V\|$	$\|E\|$	input	hidden	output	(in CSR)	3-layer GCN	3-layer GAT	3-layer GCN	3-layer GAT
it-2004 [25]	41M	1.2B	256	128	64	13.5 GB	39.4 GB	19.7 GB	128.0 GB	151.1 GB
ogbn-paper [8]	111M	1.6B	128	128	172	18.3 GB	52.9 GB	71.1 GB	335.9 GB	478.1 GB
igb260m [11]	269M	4.0B	128	128	19	45.7 GB	128.4 GB	128.4 GB	661.2 GB	853.4 GB

Background-What We'll Discuss Next

- Problems & Paper Tables

Papers \ Problems	Partitioning	Workload Unbalance	Memory Manager
Neugraph	O	O	O
ROC	O	O	O
GNNAdvisor	O	O	X
FlexGNN	X	X	O

NeuGraph: Parallel Deep Neural Network Computation on Large Graphs

Lingxiao Ma and Zhi Yang, Peking University; Youshan Miao, Jilong Xue,
Ming Wu, and Lidong Zhou, Microsoft Research; Yafei Dai, Peking University

USENIX 2019

Papers \ Problems	Partitioning	Workload Unbalance	Memory Manager
Neugraph	O	O	O

NeuGraph - SAGA-NN Model

- **What is SAGA-NN Model?**
  - Model proposed to express diverse GNNs under a single execution abstraction
  - It breaks each GNN layer into four stages

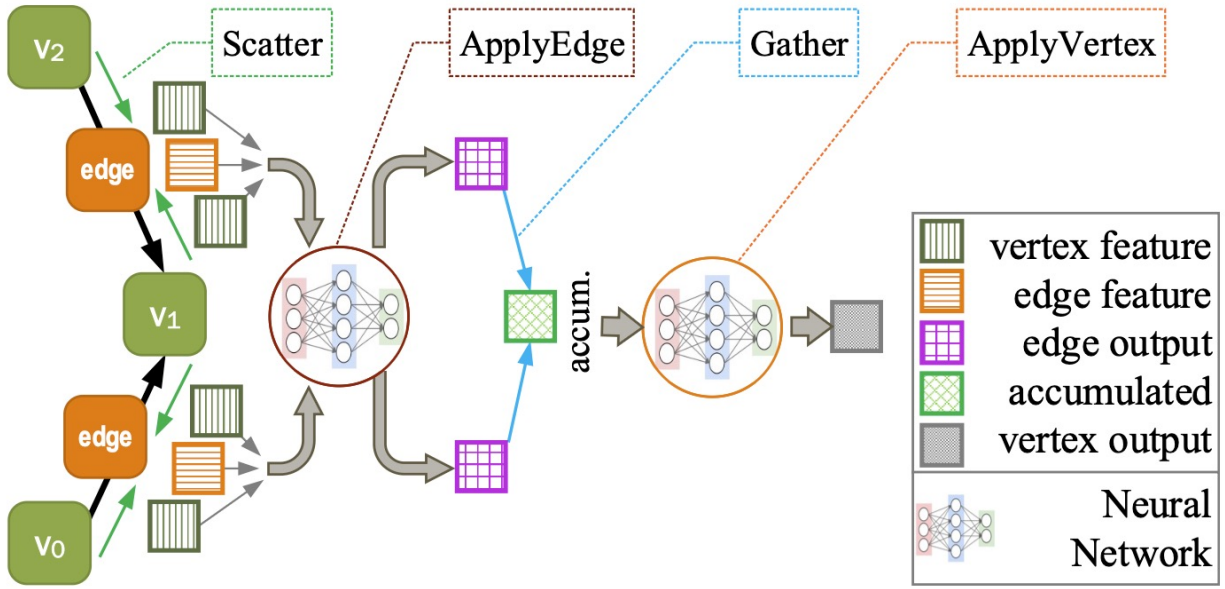
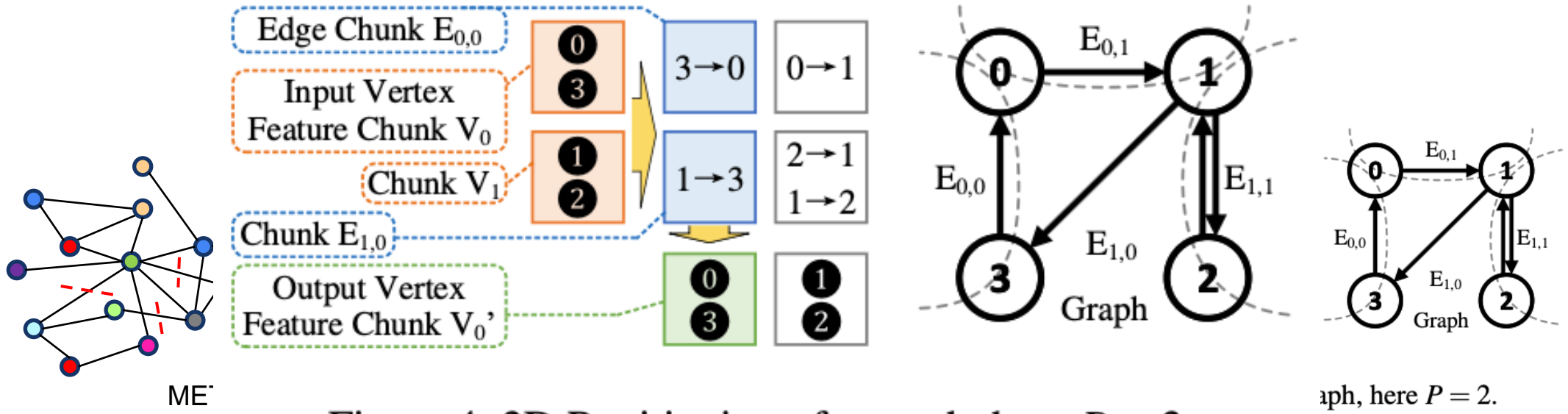


Figure 2: SAGA-NN stages for each layer of GNN.

NeuGraph – Partitioning

- **How does NeuGraph perform partitioning?**
  - Use the METIS partitioning algorithm to split the graph into subgraphs
  - Further divide the 2D graph-partitioned subgraphs into chunks



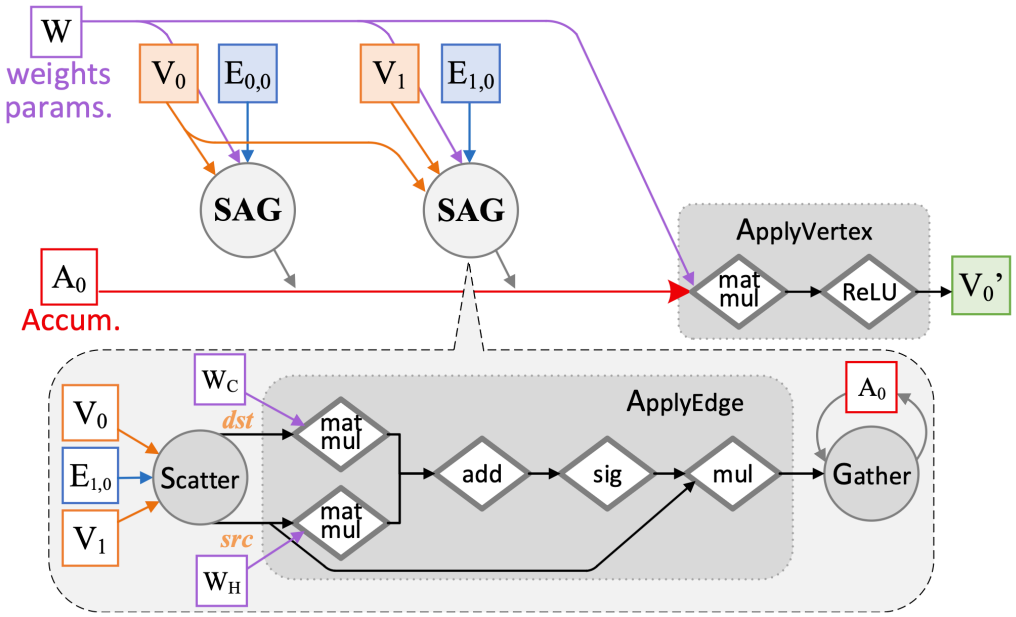
ME

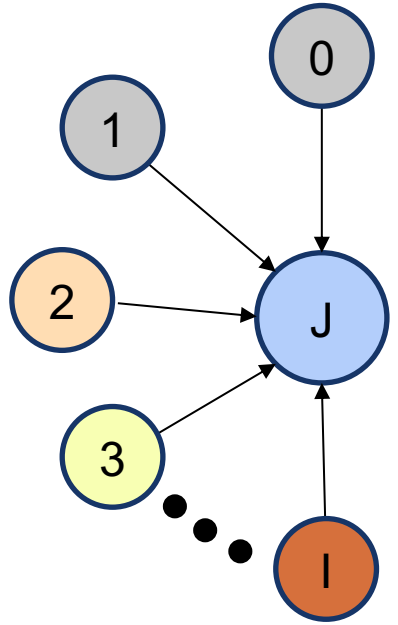
aph, here $P = 2$.

Figure 4: 2D Partitioning of a graph, here $P = 2$.

NeuGraph - Graph-Aware Dataflow Translation

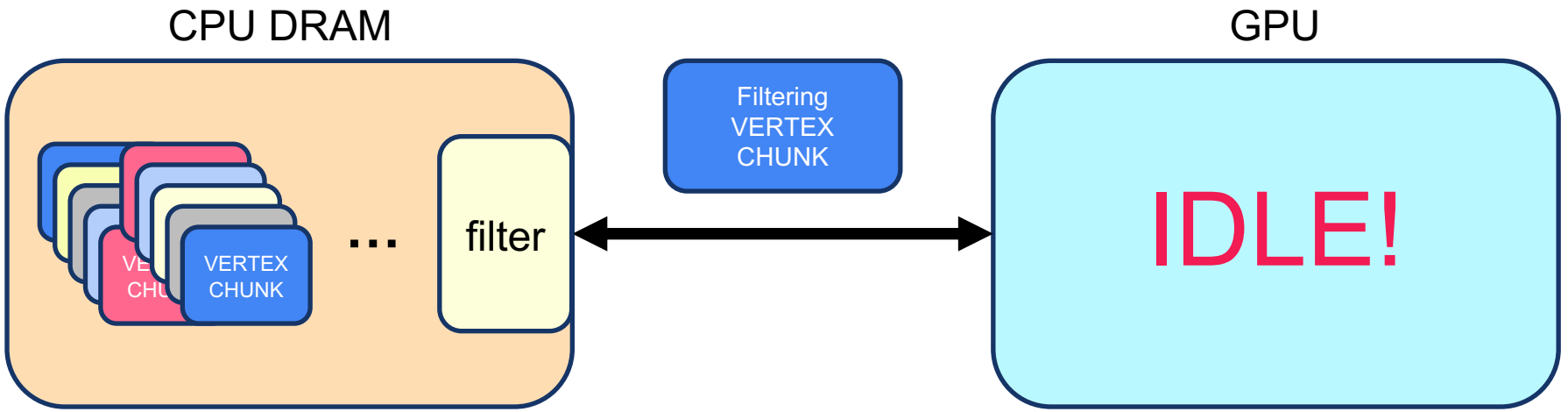
- **Forward Pass Translation**
  - In column-wise scheduling, the destination vertex chunk V_j is fixed.
  - The edge chunks in that column, $\{E_{ij}, E_{1j}, \ldots, E_{P-1j}\}$, are executed sequentially
  - Keep V_i and the accumulator on the GPU, and load each V_i and its edge chunk(s) per step





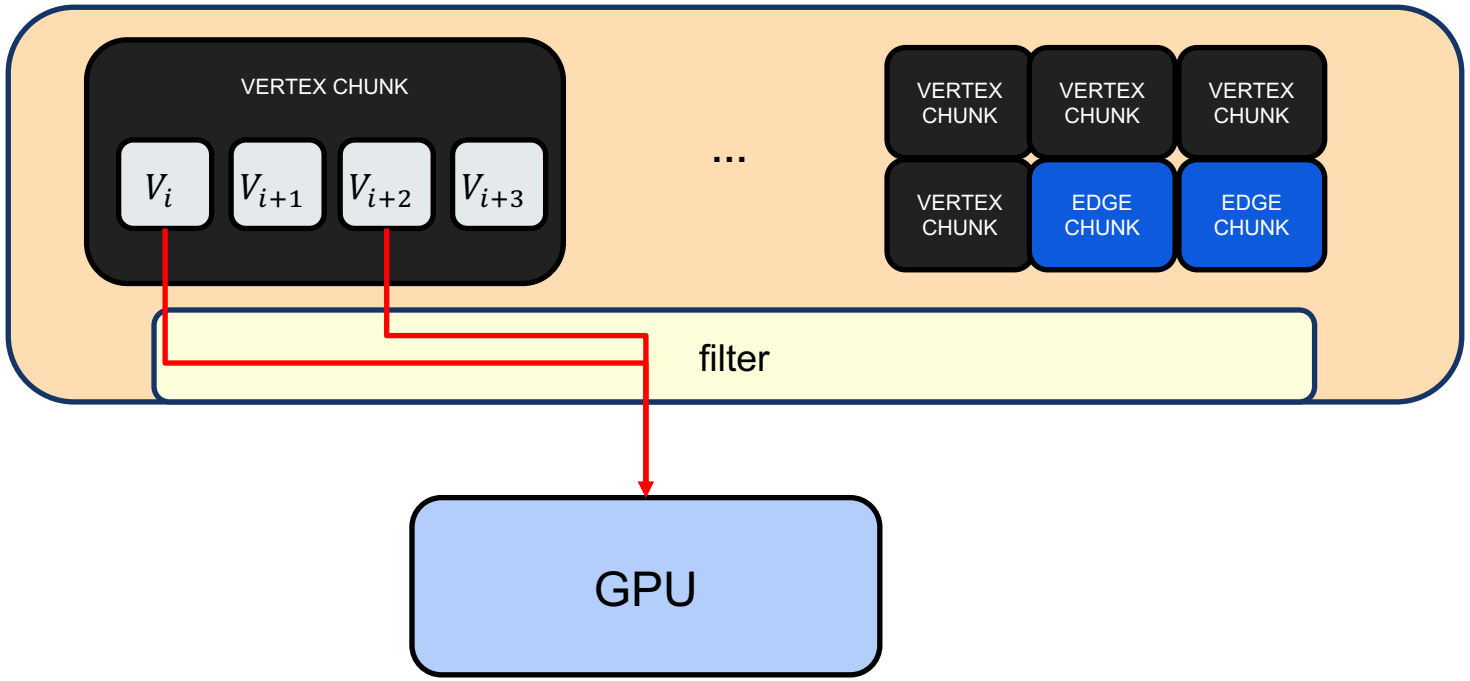
NeuGraph - Streaming Processing out of GPU Core

- **Issues Before Scheduling**
  - If H2D (host-to-device) transfer and GPU computation run sequentially, the GPU becomes idle during transfers
  - Copying vertex data to the GPU requires heavy I/O bandwidth



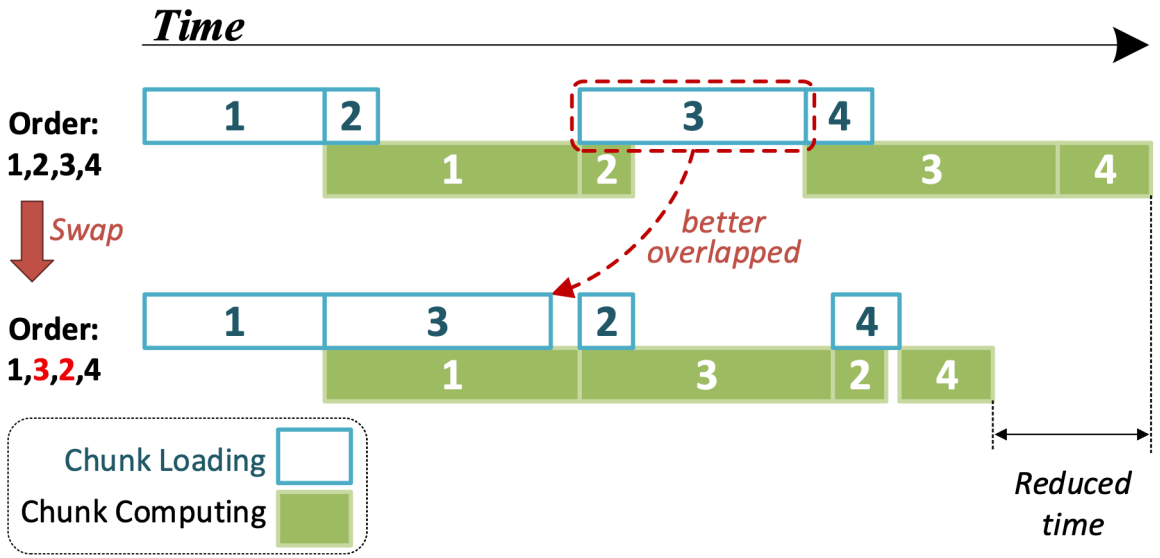
NeuGraph - Streaming Processing out of GPU Core

- **Selective Scheduling**
  - Apply filtering $\theta < \frac{T_{copy}}{T_{copy}+T_{trans}}$
  - $Cost_{filter} < Cost_{non\ filter} \rightarrow \frac{\theta}{T_{copy}} + \frac{\theta}{T_{trans}} < \frac{1}{T_{trans}} \rightarrow \theta\left(\frac{T_{trans}}{T_{copy}} + 1\right) < 1 \rightarrow \theta < \frac{T_{copy}}{T_{copy}+T_{trans}}$



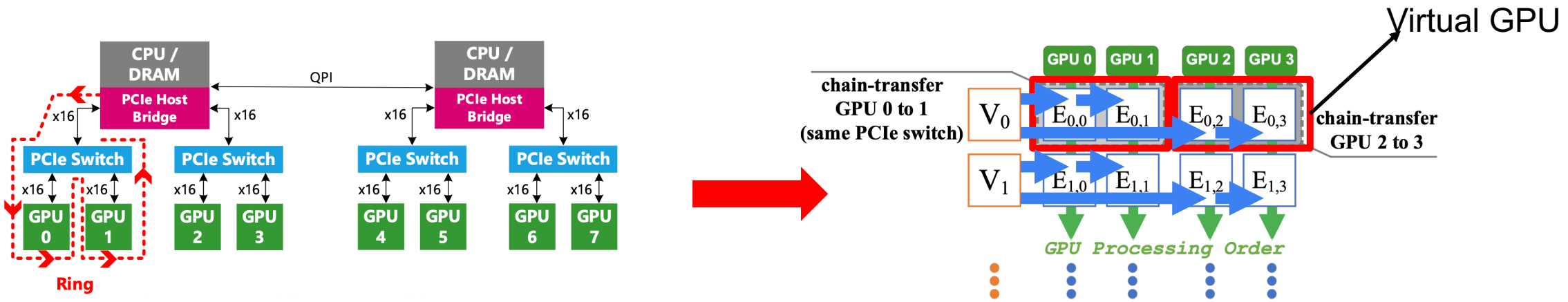
NeuGraph - Streaming Processing out of GPU Core

- **Pipeline Scheduling**
  - NeuGraph can stream many chunks through GPU memory.
    - Unlike vertex-access I/O where larger chunks helped, here smaller chunks work better (despite seeming to conflict with reducing I/O calls).
  - It further splits each edge chunk and its associated source vertex chunk horizontally into k sub-chunks.
    - k is the number of chunks that can be streamed.
    - Swapping happens only among sub-chunks split from the same edge chunk



NeuGraph-Parallel Multi GPU Processing

- **Problems**
  - Multiple GPUs share the same upstream PCIe link, so simultaneous reads from host memory can become a bottleneck
  - In GNN forward/backward propagation, all GPUs may require the same vertex chunk at certain steps
- **Chain-based Streaming Scheduling**
  - The shared vertex chunk is transferred from the host to the first GPU in the chain only once
  - It is then relayed to the other GPUs in the chain via GPU-to-GPU peer-to-peer (P2P) transfers



Improving the accuracy, scalability, and performance of Graph Neural Networks with ROC

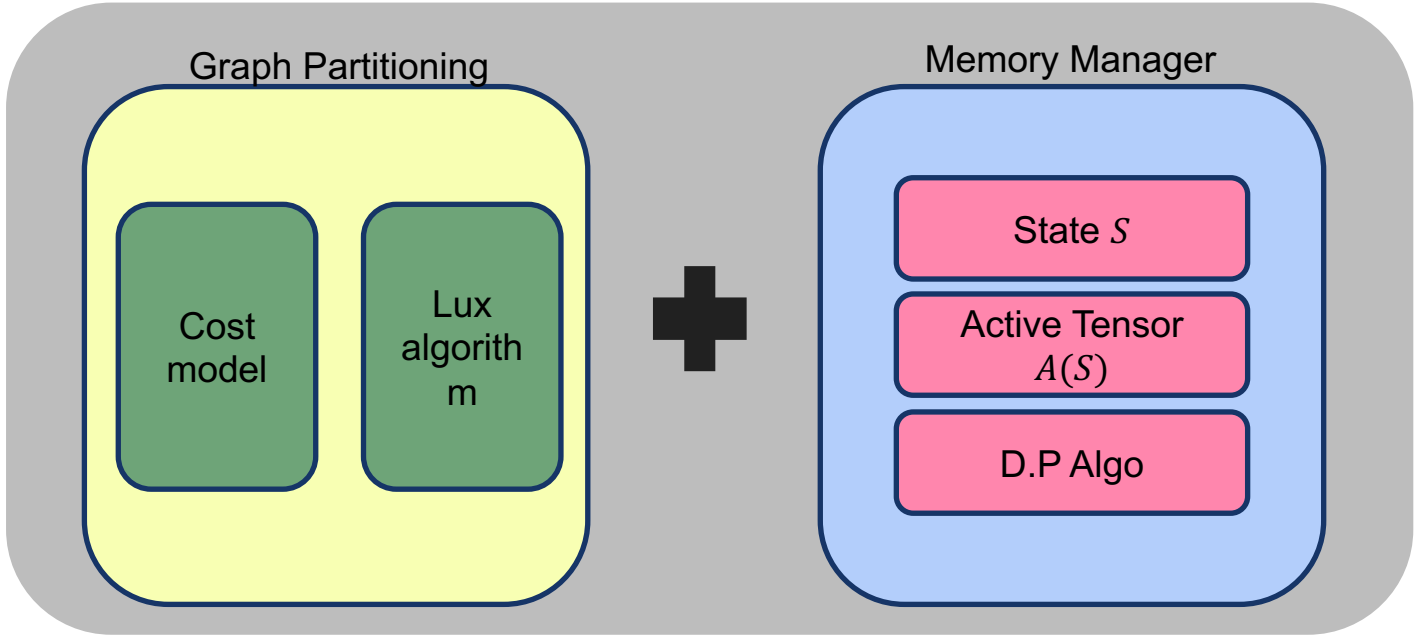
Zhihao Jia Sina Lin Mingyu Gao Matei Zaharia Alex Aiken

Proceedings of Machine Learning and Systems, 2020

Papers \ Problems	Partitioning	Workload Unbalance	Memory Manager
ROC	O	O	O

ROC-Overview

- **Architect**



- **Overview**
  - Multi compute node environments

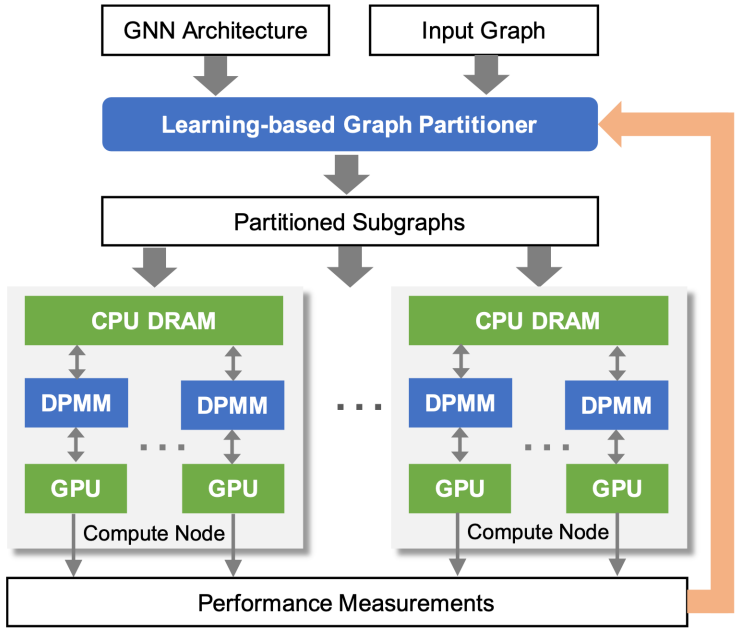
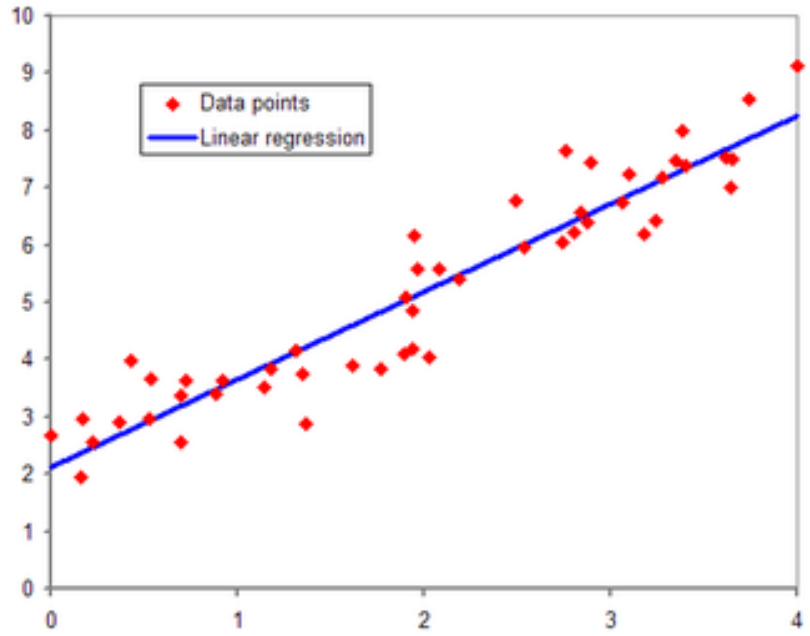


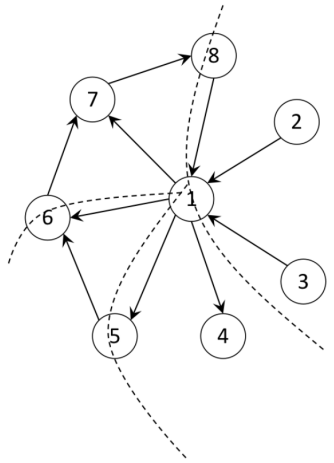
Figure 2. ROC system overview. DPMM represents dynamic-programming-based memory manager.

ROC-Graph Partitioner

- **Cost Model**
  - Online-linear-regression-based model
  - $y = \omega_0 x_0 + \omega_1 x_1 + \cdots$



- **Partitioning Algo**
  - Lux Algo



Edges

Partition A

1	2
1	3
1	8

Partition B

4	1
5	1

Partition C

6	1
6	5

Partition D

7	1
7	6
8	7

In-Neighbor Sets

	Node	INS
A	1	{2, 3, 8}
B	1	{1}
C	2	{1, 5}
D	2	{1, 6, 7}

Out-Neighbor Sets

	Node	ONS
A	1	{1}
B	1	{4, 5}
C	2	{6}
D	2	{7, 8}

Update Sets

Node	UDS
1	{2, 3, 8}
2	{1, 5}

ROC-Graph Partitioner

- **Cost Model**
  - Online-linear-regression-based model

	Definition	Description
x_1	1	the vertex itself
x_2	$\lvert\mathcal{N}(v)\rvert$	number of neighbors
x_3	$\lvert\mathcal{C}(v)\rvert$	continuity of neighbors
x_4	$\sum_i \lceil \frac{c_i(v)}{\mathtt{WS}} \rceil$	# mem. accesses to load neighbors
x_5	$\sum_i \lceil \frac{c_i(v) \times d_{in}}{\mathtt{WS}} \rceil$	# mem. accesses to load the activations of all neighbors

Input dimension

$$t(l,v) = \sum_i w_i(l)x_i(v)$$

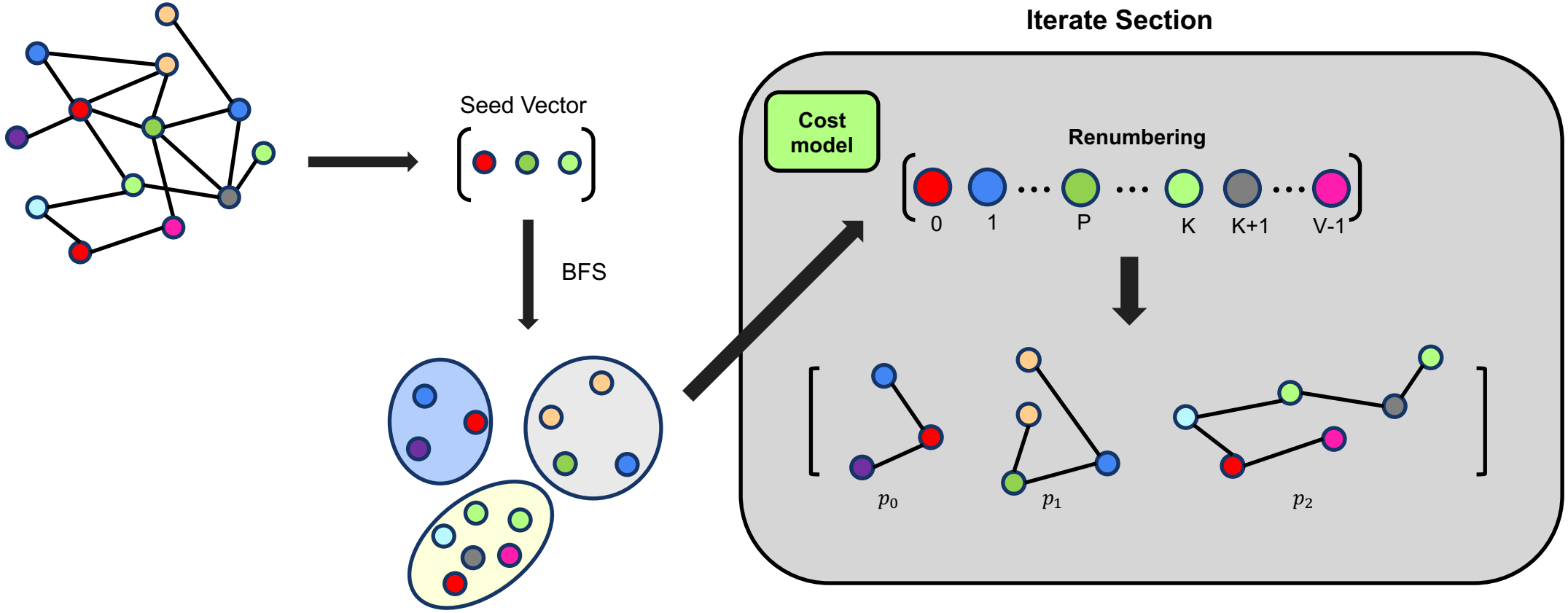
$$t(l,\mathcal{G}) = \sum_{v\in\mathcal{G}} t(l,v) = \sum_{v\in\mathcal{G}}\sum_i w_i x_i(v) = \sum_i w_i \sum_{v\in\mathcal{G}} x_i(v) = \sum_i w_i x_i(\mathcal{G})$$

$C(v) = \{c1(v), \ldots, c_{|C|}(v)\}$
$c_i(v)$: *range of consecutively numbered vertices*
$v_1's$ *neighbor* $\rightarrow \{v_3, v_4, v_6, v_8\}$
$c_1(v_1) = \{v_3, v_4\}, c_2(v_1) = \{v_6\}, c_3(v_1) = \{v_8\}$

$$Loss(l) = \frac{1}{N}\sum_{i=1}^{N}\left(t(l,\mathcal{G}_i) - y(l,\mathcal{G}_i)\right)^2$$

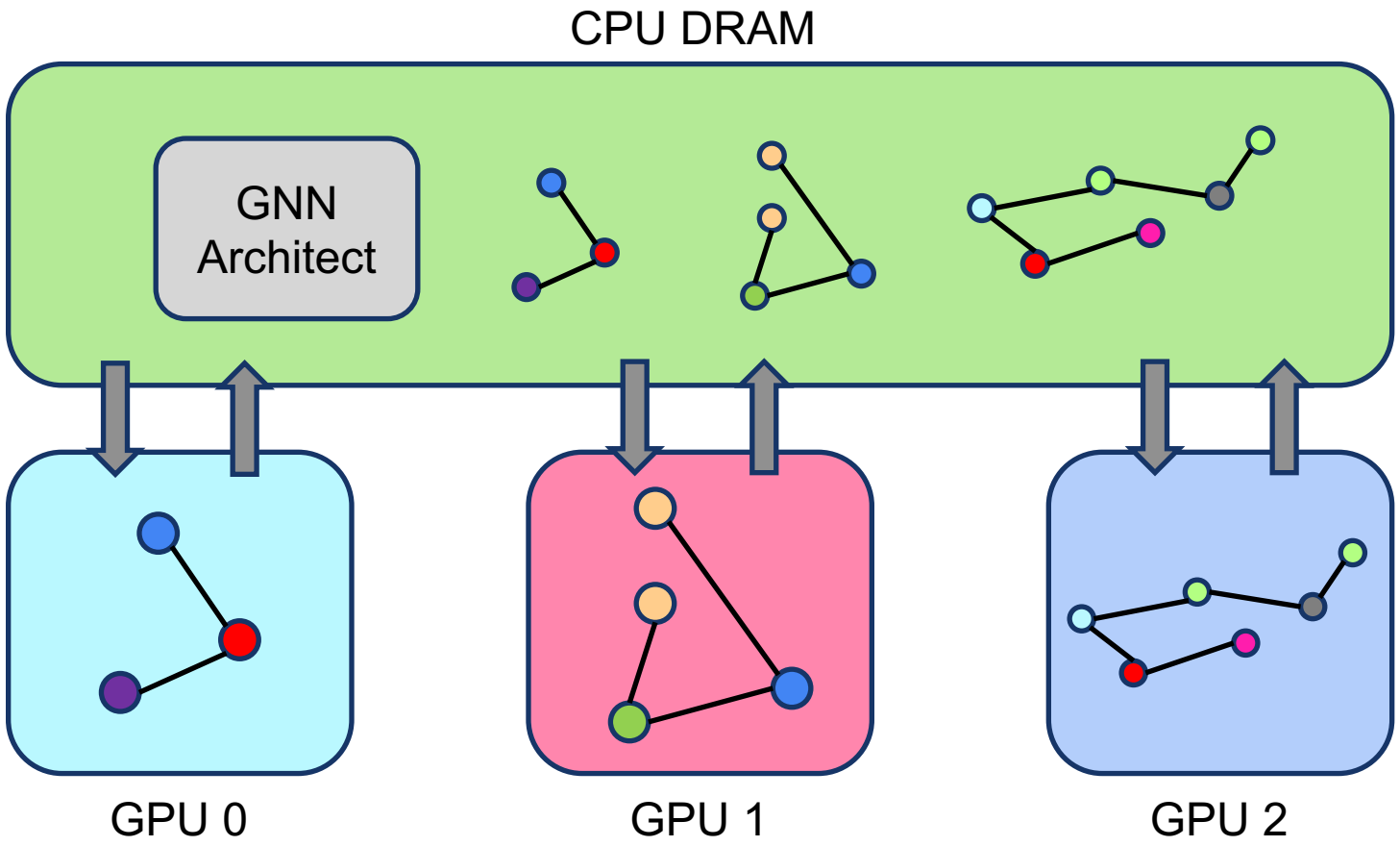
ROC-Graph Partitioner

- **Lux Algorithm + Cost Model**



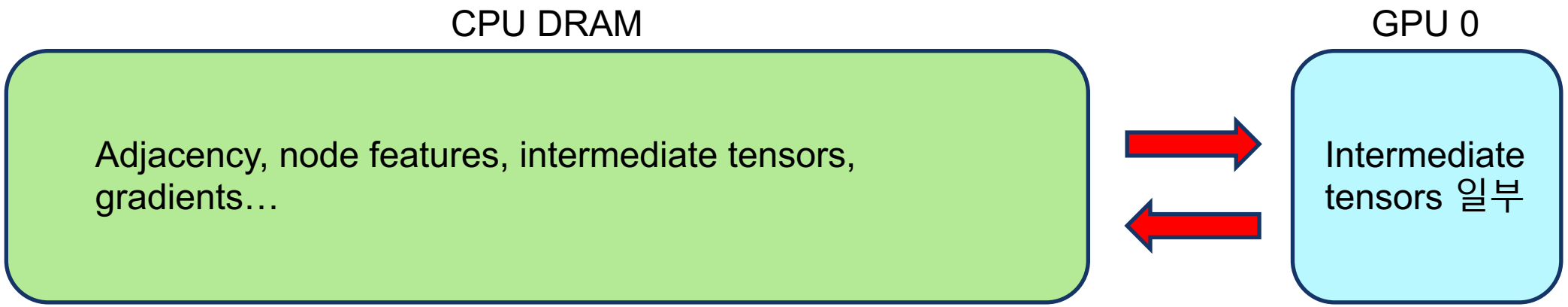
ROC-Memory Manager

- **Memory Manager**



ROC-Memory Manager

- **Inside of GPU**
  - ROC All GNN computations are performed on the GPU
  - It is difficult and unnecessary to load all GNN data onto the GPU
- **Idea**
  - GNN data is stored in CPU DRAM
  - The GPU caches only the necessary tensors



ROC-Memory Manager

- **Valid State**
  - The set of operations completed so far

- **Activation Tensors**
  - Reusable tensors

Table 3. All the valid states and their activation tensors for the GNN architecture in Figure 3.

Valid State $\mathcal{S}$	**Activation Tensors $\mathcal{A}(\mathcal{S})$**
$\{①\}$	$\{g, a\}$
$\{①, ②\}$	$\{g, a, b, w_1\}$
$\{①, ②, ③\}$	$\{g, a, b, h^1, w_1, w_2\}$
$\{①, ②, ③, ④\}$	$\{g, a, b, w_1, w_2, \nabla L(h^1)\}$
$\{①, ②, ③, ④, ⑤\}$	$\{g, a, b, w_1, \nabla L(b)\}$
$\{①, ②, ③, ④, ⑤, ⑥\}$	$\{g, a, \nabla L(a)\}$
$\{①, ②, ③, ④, ⑤, ⑥, ⑦\}$	$\{\}$

ROC-Memory Manager

- **Example**

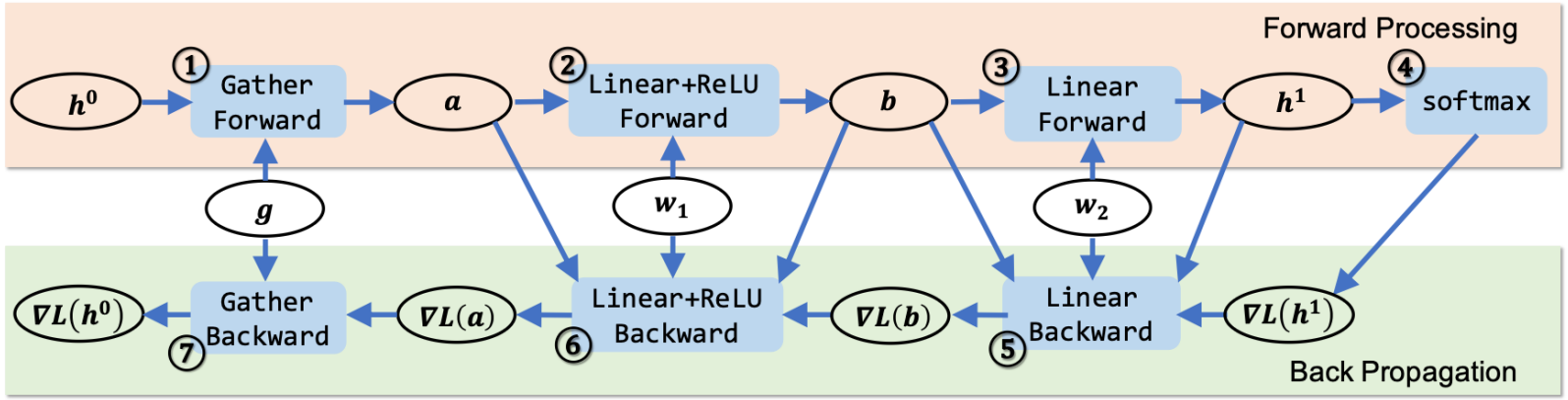


Table 3. All the valid states and their activation tensors for the GNN architecture in Figure 3.

Valid State $\mathcal{S}$	**Activation Tensors $\mathcal{A}(\mathcal{S})$**
{①}	$\{g, a\}$
{①, ②}	$\{g, a, b, w_1\}$
{①, ②, ③}	$\{g, a, b, h^1, w_1, w_2\}$
{①, ②, ③, ④}	$\{g, a, b, w_1, w_2, \nabla L(h^1)\}$
{①, ②, ③, ④, ⑤}	$\{g, a, b, w_1, \nabla L(b)\}$
{①, ②, ③, ④, ⑤, ⑥}	$\{g, a, \nabla L(a)\}$
{①, ②, ③, ④, ⑤, ⑥, ⑦}	$\{\}$

ROC-Memory Manager

- **Algorithm**
  - Dynamic programming
  - Greedy, brute force… are not appropriate

Algorithm 1 A recursive dynamic programming algorithm for computing minimum data transfers. $\textsc{In}(o_i)$ and $\textsc{Out}(o_i)$ return the input and output tensors of the operation o_i, respectively, and size($\mathcal{T}$) returns the memory space required to save all tensors in $\mathcal{T}$.

Input: An input graph g, a GNN architecture $\mathcal{G}$, and the GPU device memory capacity *cap*.
Output: Minimum data transfers required to compute $\mathcal{G}$ on g within capacity *cap*.
▷ $\mathcal{D}$ is a database storing all computed $\textsc{Cost}$ functions.

function $\textsc{Cost}(\mathcal{S}, \mathcal{T})$
  if $(\mathcal{S}, \mathcal{T}) \in \mathcal{D}$ **then**
    return $\mathcal{D}(\mathcal{S}, \mathcal{T})$
  if $\mathcal{S}$ is $\varnothing$ **then**
    return size($\mathcal{T}$)
  $cost \leftarrow \infty$
  for $o_i \in \mathcal{S}$ **do**
    if $(\mathcal{S} \setminus o_i)$ is a valid state **then**
      $\mathcal{S}' \leftarrow \mathcal{S} \setminus o_i$
      $\mathcal{T}' \leftarrow \big(\mathcal{T} \setminus \textsc{Out}(o_i)\big) \cap \mathcal{A}(\mathcal{S}')$
      $xfer \leftarrow \text{size}\big(\textsc{In}(o_i) \setminus \mathcal{T}'\big)$
      if $\text{size}\big(\mathcal{T} \cup \textsc{In}(o_i) \cup \textsc{Out}(o_i)\big) \leq cap$ **then**
        $cost = \min\{cost, \textsc{Cost}(\mathcal{S}', \mathcal{T}') + xfer\}$
  $\mathcal{D}(\mathcal{S}, \mathcal{T}) \leftarrow cost$
  return $\mathcal{D}(\mathcal{S}, \mathcal{T})$

GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs

Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie,
and Yufei Ding, University of California, Santa Barbara

USENIX 2021

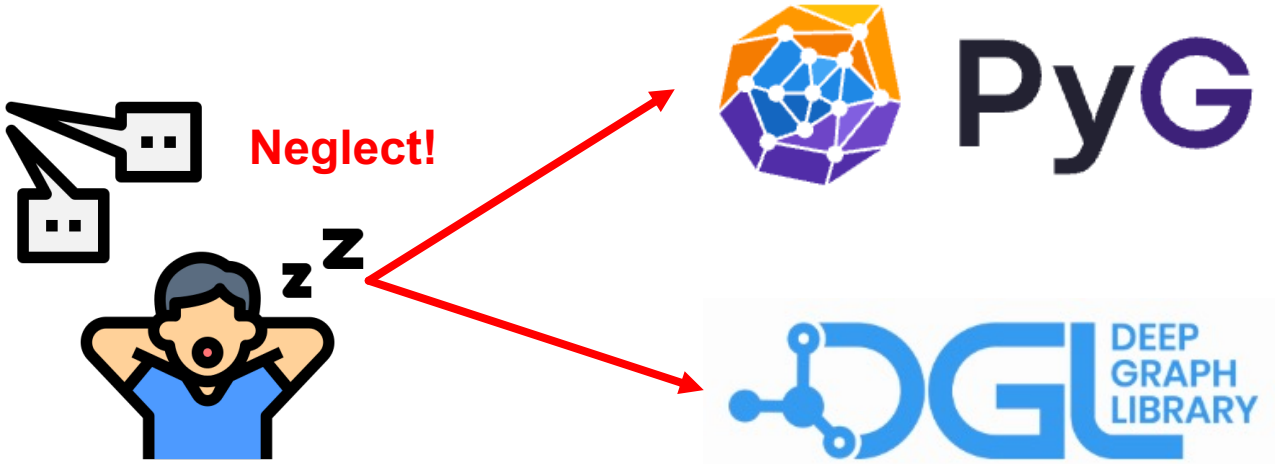
Papers \ Problems	Partitioning	Workload Unbalance	Memory Manager
GNNAdvisor	O	O	X

Background

- **One Size Fits All GNN Frameworks**
  - Same inference
  - Same optimization pattern
  - Existing frameworks like PyG and DGL always use the same static CUDA kernel for various GNNs

- **Optimizations not tailored to GNN**
  - Existing GNN frameworks simply extend the optimization schemes from classical graph systems
  - Do not address the difference between GNN and graph processing

GIN, GCN, GraphSAGE…
Existing GNN's characteristics



GNNAdvisor

- **Overview**
  - Loader & Extractor
  - Decider
  - Kernel & Runtime Crafter

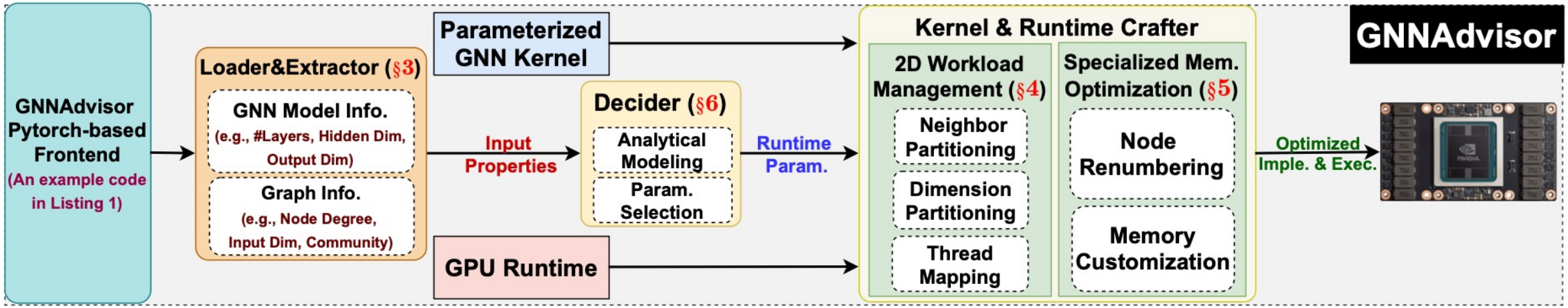
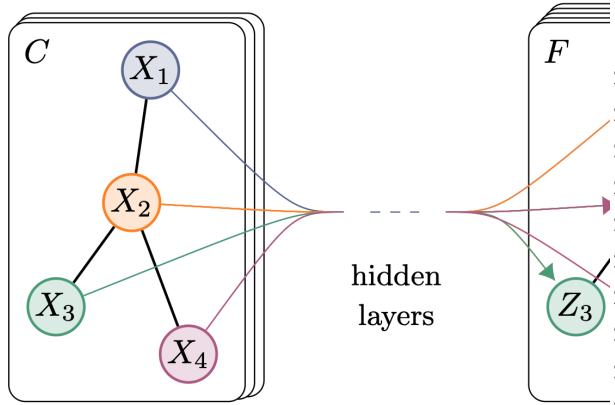


Figure 1: Overview of GNNAdvisor.

GNNAdvisor

- **Input Analysis & GNN Model I**
  - GNN input information can guid
  - That different GNN application s
  - Different Aggregation methods a
  - GNNAdvisor uses PyTorch as it

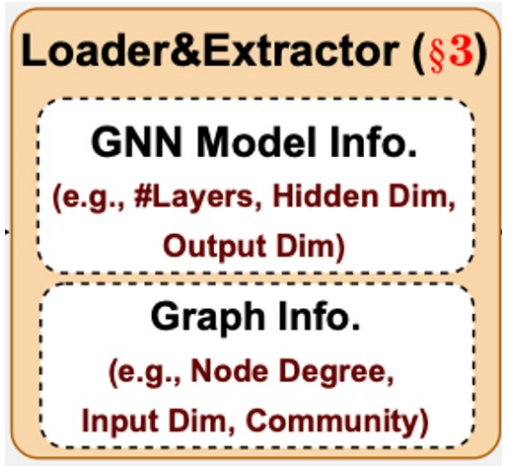


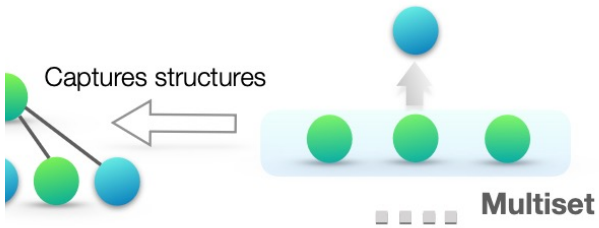
(a) Graph Convolutional Ne

Listing 1: Example of a 2-layer GCN in GNNAdvisor.

```python
import GNNAdvisor as GNNA
import torch
# import other packages ...

# Create a GCN class.
class GCN(torch.nn.Module):
    def __init__(self, inDim, hiDim, outDim, nLayers):
        self.layers = torch.nn.ModuleList()
        self.layers.append(GNNA.GCNConv(inDim, hiDim))
        for i in range(nLayers - 2):
            layer = GNNA.GCNConv(hiDim, hiDim)
            self.layers.append(layer)
        self.layers.append(GNNA.GCNConv(hiDim, outDim))
        self.softmax = torch.nn.Softmax()

    def forward(self, X, graph, param):
        for i in range(len(self.layers)):
            X = self.layers[i](X, graph, param)
            X = self.ReLU(X)
        X = self.softmax(X)
        return X

# Define a two-layer GCN model.
model = GCN(inDim=100, hiDim=16, outDim=10, nLayers=2)

# Loading graph and extracting input propertities.
graphObj, inputInfo = GNNA.LoaderExtractor(graphFile,
                                           model)
# Set runtime parameters automatically.
X, graph, param = GNNA.Decider(graphObj, inputInfo)

# Run model.
predict_y = model(X, graph, param)

# Compute loss and accuracy.
# Gradient backpropagation for training.
```

tor



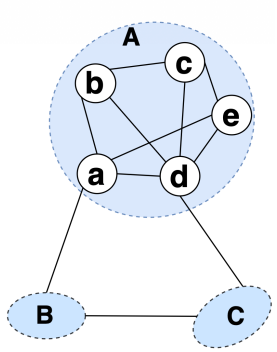


ee

GNN aggregation

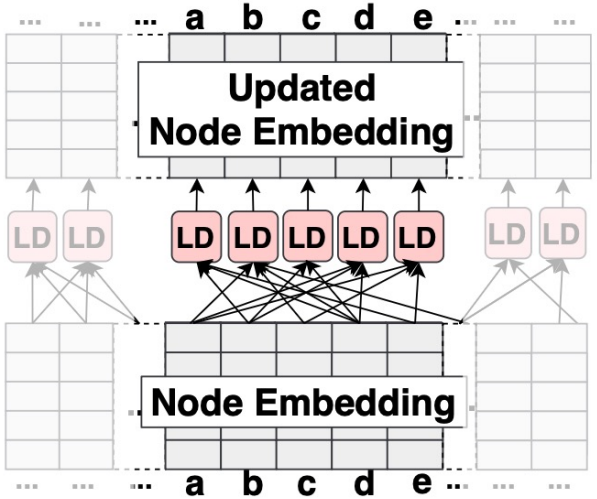
GNNAdvisor

- **Graph Community**
  - Community inside (edge density ↑)
  - Community outside (edge density ↓)



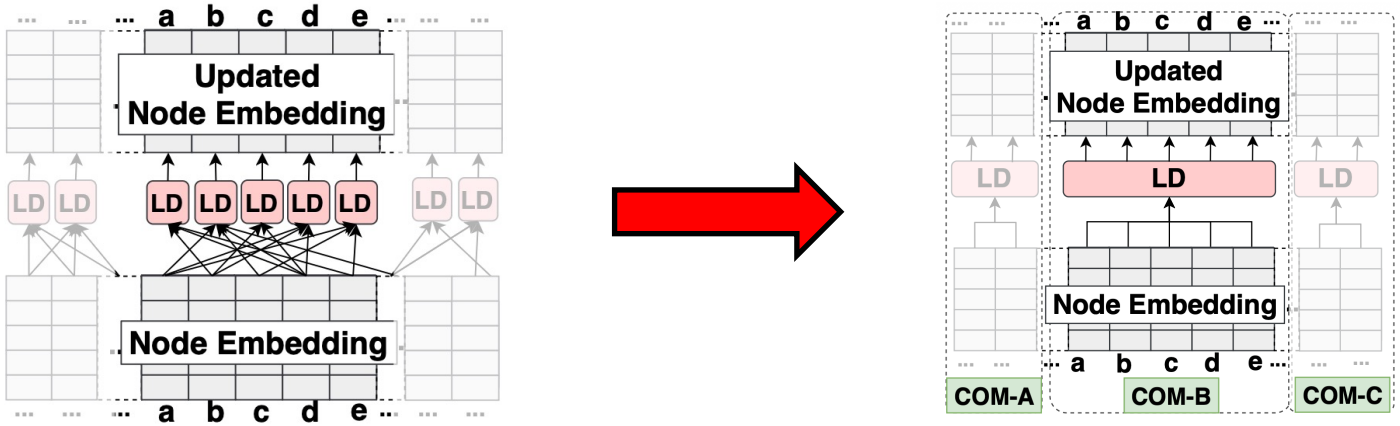
(a) Graph Community

- **Node-centric aggregation**
  - First load its neighbors and then do aggregation independently
  - Can achieve great computation parallelism when each neighbor has a lightweight scalar attribute
    - However in GNN node is vector
    - GNN node embedding is large



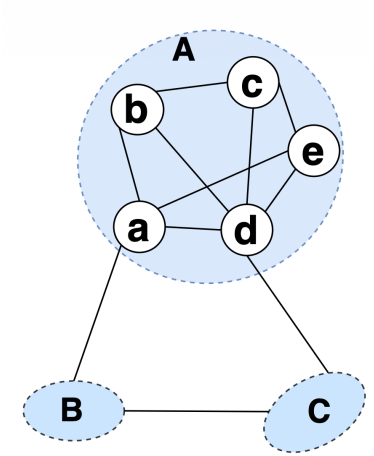
GNNAdvisor

- **Use “COMMON” neighbors!**
  - When performing aggregation independently, it does not utilize the concept of common neighbors
    - Situations where one is loaded multiple times occur
  - From the entire system perspective, redundant situations occur where the same neighbors are read multiple times
  - Identify the shared neighbor set and load it only once



GNNAdvisor

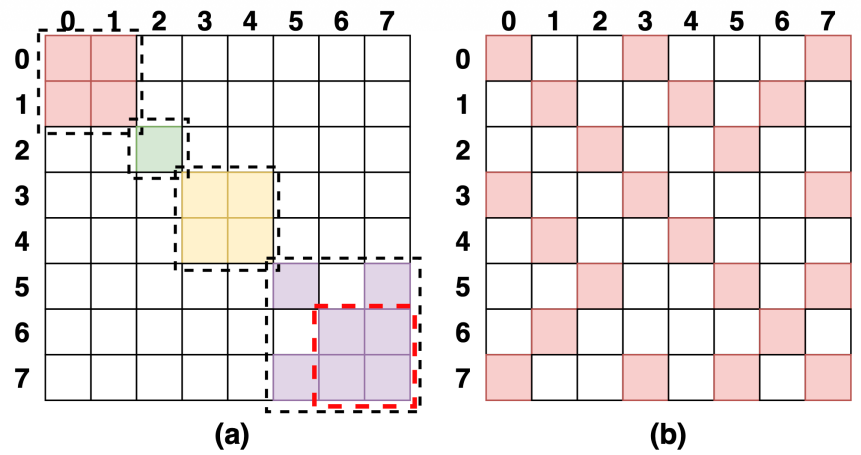
- **Key Idea!**
  - Existing methods are CPU-specific
    - CPU and GPU are fundamentally different, necessitating the development of distinct approaches
    - CPU → MB-level cache per thread, GPU → L1 = KB-level cache shared per SM unit
  - To leverage community locality on GPU, threads/warps must be assigned to nearby thread/warp IDs
    - Need to capture the communities of a graph and map such locality from node-ID adjacency to GPU kernel adjacency



(a) Graph Community

GNNAdvisor

- **Community-aware Node Renumbering**
  - Enhance data locality using the graph community, nodes within the same community must have close node IDs even on the GPU
    - Renumber the nodes!
  - Renumbering on all nodes is wasteful
    - If the situation is like a, it's not necessary
    - Verify if it's needed via the AES formula
    - Rabbit reordering

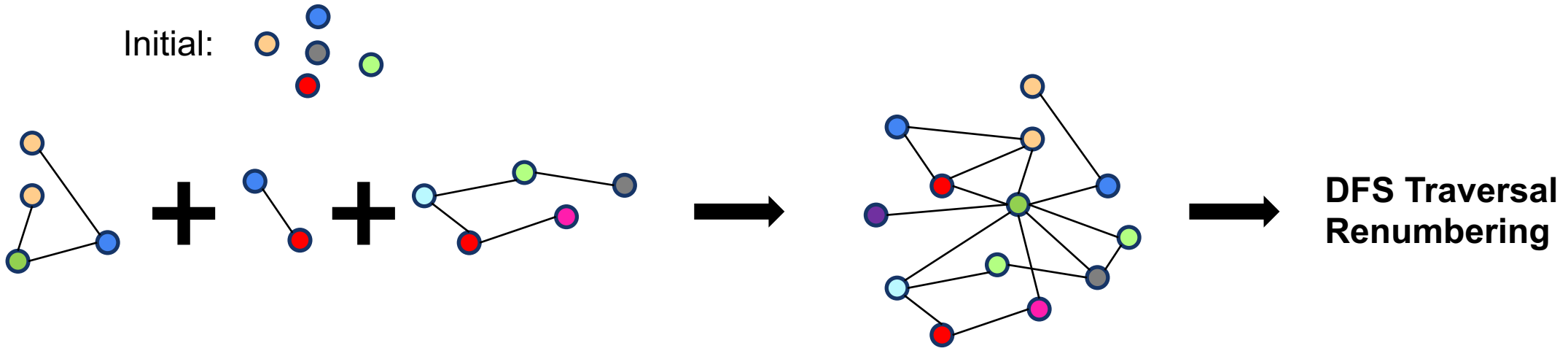


$$\mathbf{AES} = \frac{1}{\#E} \sum_{(src_{id}, trg_{id}) \in E} |src_{id} - trg_{id}|$$

$$\sqrt{AES} > \lfloor \frac{\sqrt{\#N}}{100} \rfloor$$

GNNadvisor

- **Rabbit Reordering**
  - Optimizing modularity (max modularity) based on Edge
  - Hierarchical community merging
  - Assigning IDs by traversing each community internally using DFS traversal
- **Graph Modularity**
  - Modularity is higher when there are many edges within the same community and few edges between different communities



GNNAdvisor

- **2D Workload Management**
  - GNN workloads grow in -> the number of neighbors and the size of the embedding dimension
  - Coarse-grained neighbor partitioning, fine-grained dimension partitioning, and warp based thread alignment

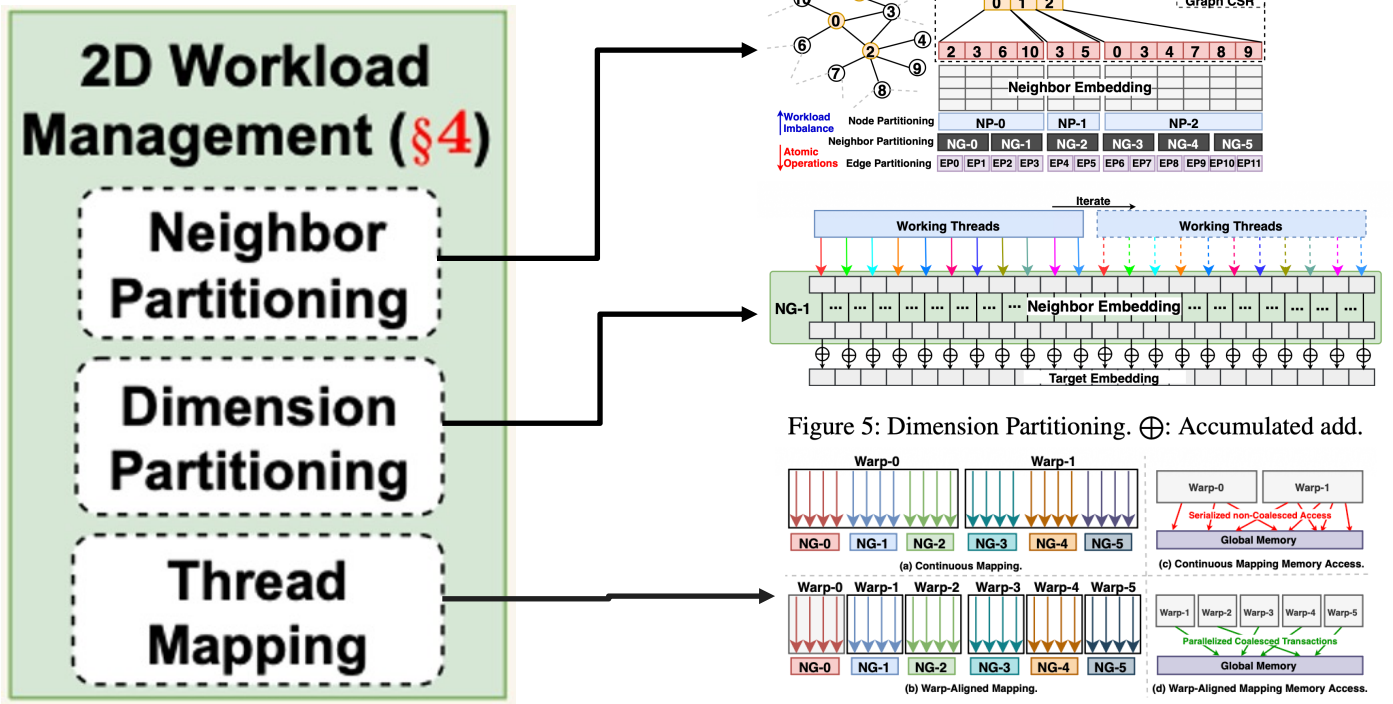
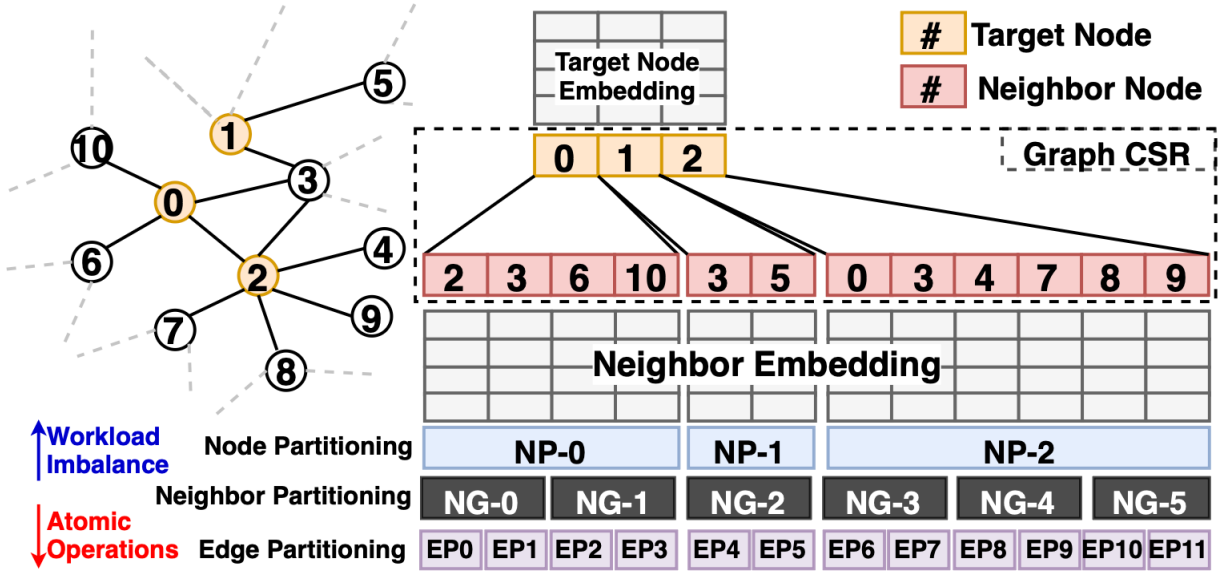


Figure 5: Dimension Partitioning. ⊕: Accumulated add.

Figure 6: Warp-based Thread Alignment.

GNNAdvisor

- **Coarse-grained Neighbor Partitioning-Low Dimension Settings**
  - Data format: CSR format
  - Neighbor partitioning can largely mitigate the size irregularity of the workload units
  - Neighbor-partitioning solution can avoid the overheads of managing many tiny workload units



GNNAdvisor

- **Fine-grained Dimension Partitioning-High Dimensional Node Embeddings**
  - Embedding dimensions are independent
  - Can accommodate a more diverse range of embedding dimension sizes
  - It introduces another performance-related parameter, dw(dimension worker)=working threads

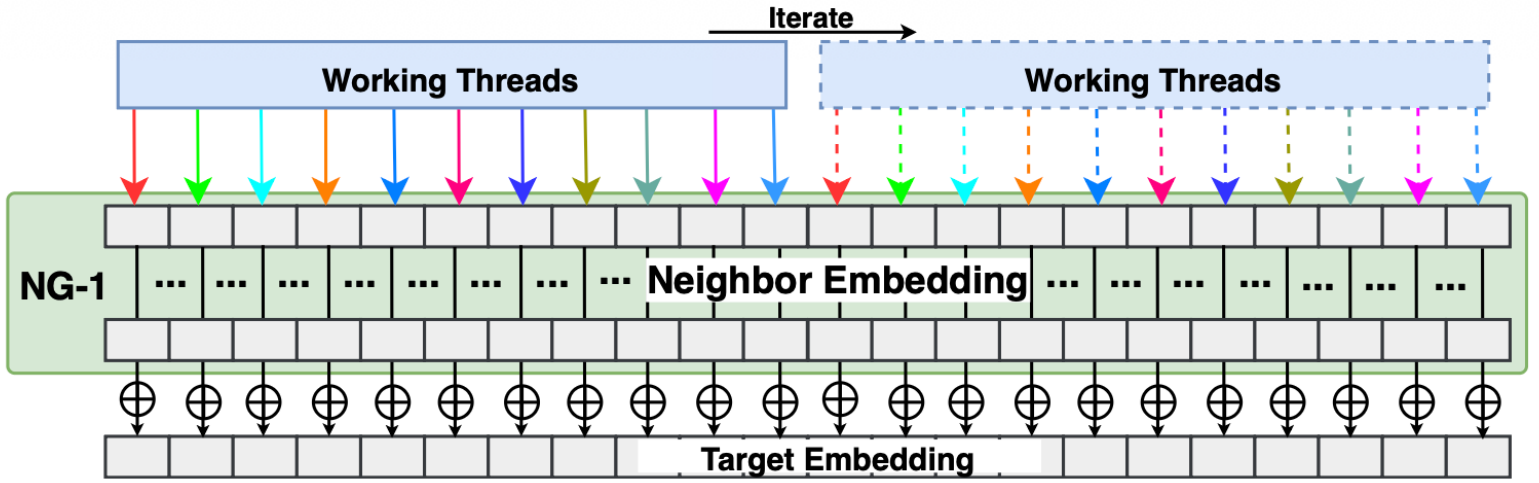


Figure 5: Dimension Partitioning. ⊕: Accumulated add.

GNNAdvisor

- **Warp-based Thread Alignment**
  - Different behaviors among these threads would result in thread divergence and GPU underutilization
  - Inter-thread synchronization (e.g., atomic operations) can be minimized
  - The workload of a single warp is reduced and different warps will process more balanced workloads
  - Memory access can be coalesced

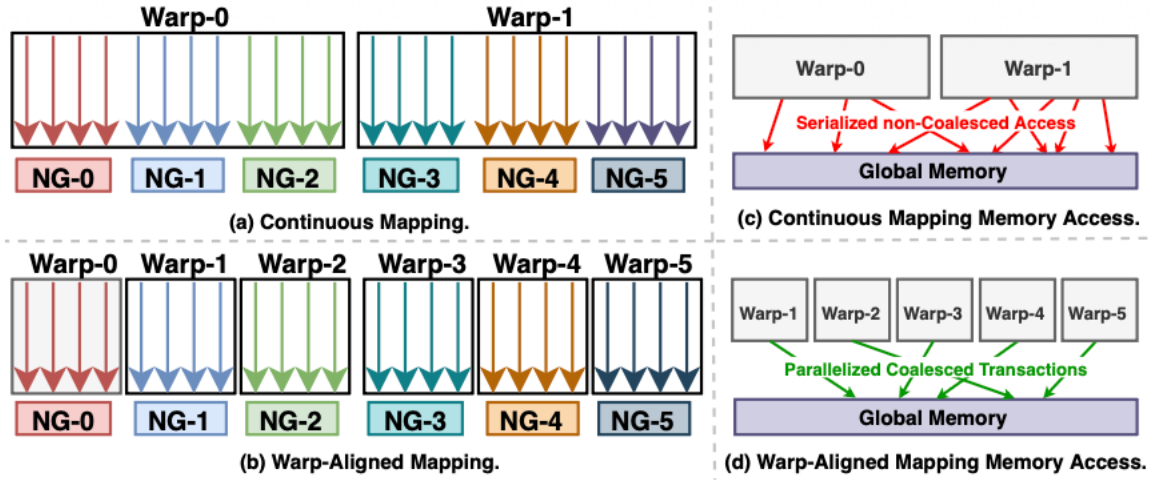
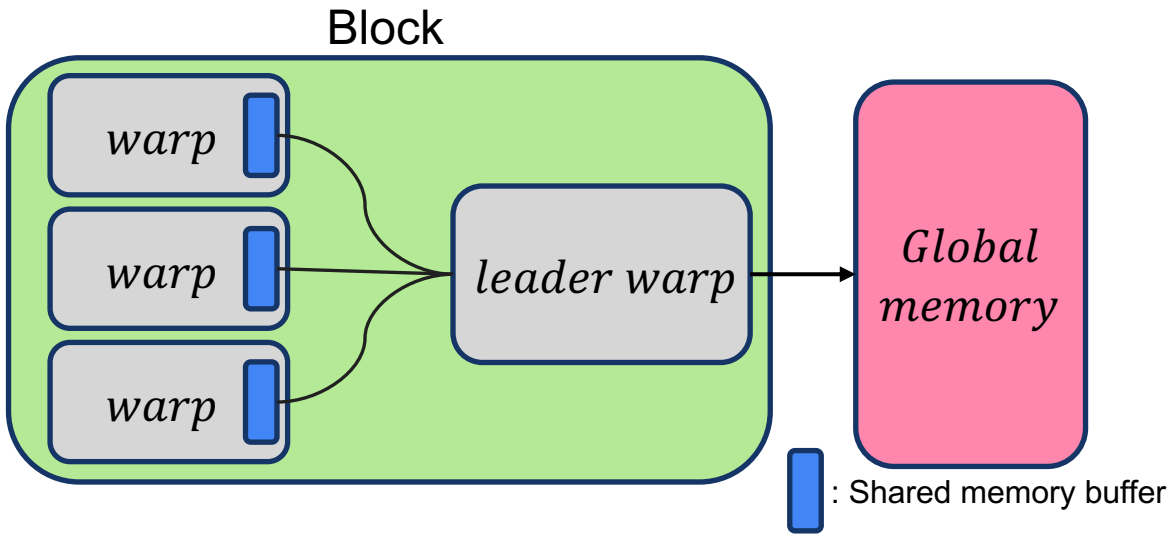
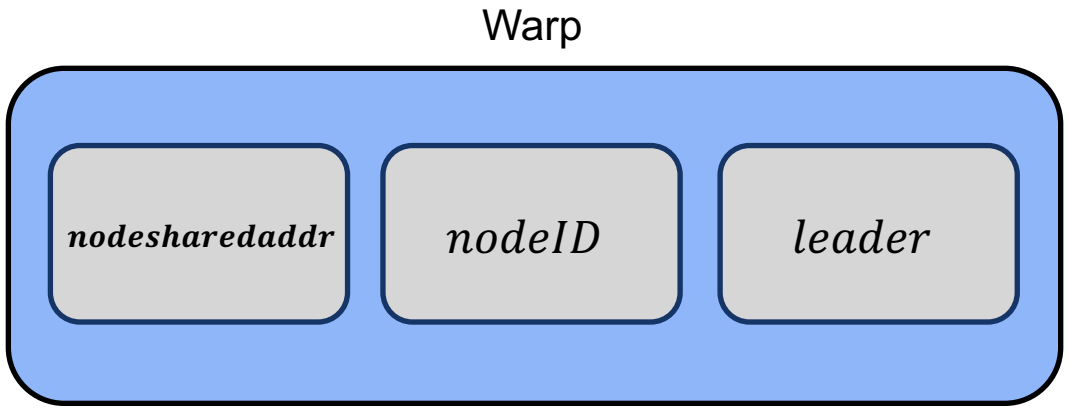


Figure 6: Warp-based Thread Alignment.

GNNAdvisor

- **Warp-aware Memory Customization**
  - Reserve a shared memory space (4×Dim bytes) for the target node of each neighbor group
  - The emergence of Leader warp – Flushing to global memory is done only once per warp
    - If each warp flushes? → Atomic ↑ $O(k * ngs * Dim)$





FlexGNN: A High-Performance, Large-Scale Full-Graph GNN System with Best-Effort Training Plan Optimization

Jeongmin Bae
jm_bae@kaist.ac.kr
KAIST
Daejeon, Republic of Korea

Donghyoung Han
dhhan@graphai.io
GraphAI
Daejeon, Republic of Korea

Min-Soo Kim*
minsoo.k@kaist.ac.kr
KAIST
Daejeon, Republic of Korea

KDD 2025

Papers \ Problems	Partitioning	Workload Unbalance	Memory Manager
FlexGNN	X	X	O

Background

- **Intermediate Data**
  - Results computed during forward propagation that are reused during backward propagation
  - Size of intermediate data is typically much larger than that of graph data and vertex data
    - Significantly impacting the scalability of GNN training

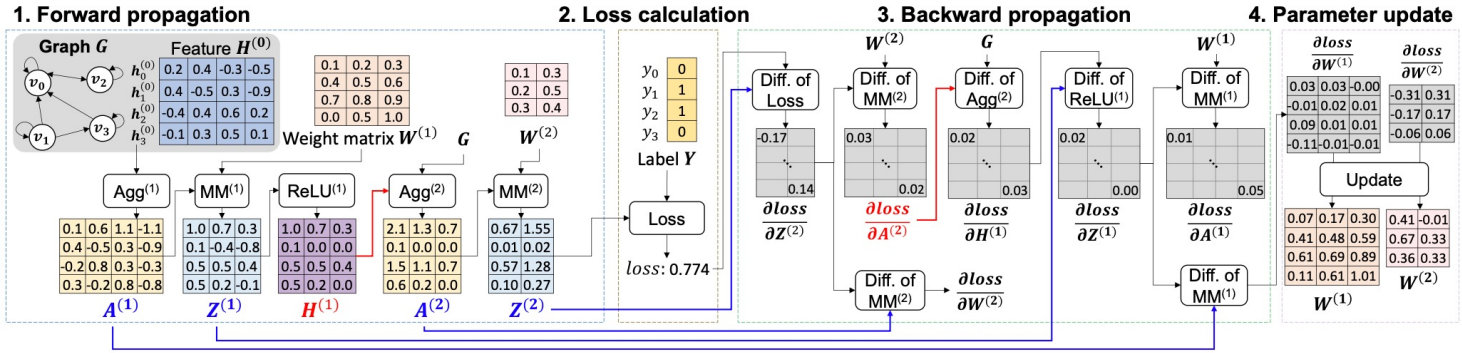


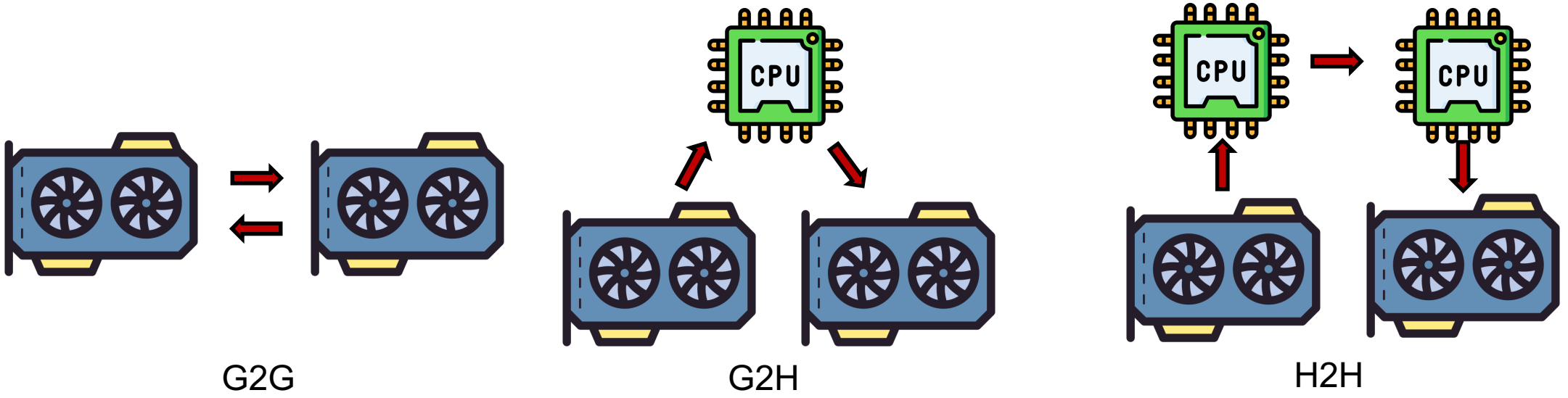
Figure 1: Intermediate data in a two-layer GCN model (Agg: aggregation, MM: matrix multiplication, Diff: differentiation).

Table 1: Sizes of graph, vertex, and intermediate data when training GNNs on real-world graphs.

Dataset			Feature dimension			Graph data	Vertex data		Intermediate data	
name	$\|V\|$	$\|E\|$	input	hidden	output	(in CSR)	3-layer GCN	3-layer GAT	3-layer GCN	3-layer GAT
it-2004 [25]	41M	1.2B	256	128	64	13.5 GB	39.4 GB	19.7 GB	128.0 GB	151.1 GB
ogbn-paper [8]	111M	1.6B	128	128	172	18.3 GB	52.9 GB	71.1 GB	335.9 GB	478.1 GB
igb260m [11]	269M	4.0B	128	128	19	45.7 GB	128.4 GB	128.4 GB	661.2 GB	853.4 GB

Background

- **Synchronous Inter GPU Communication**
  - Existing methods rely solely on a single type of synchronous inter-GPU communication
  - Three types of inter-GPU communication
    - GPU-to-GPU communication within a machine (G2G)
    - GPU to Host (main) memory and back to GPU within a machine (G2H)
    - GPU to Host memory, through the network, and back from Host memory to GPU (H2H)



FlexGNN-Training Execution Plan

- **Training Execution Plan**
  - Vanila plan
  - Lifetime of intermediate data
  - Space of training plans

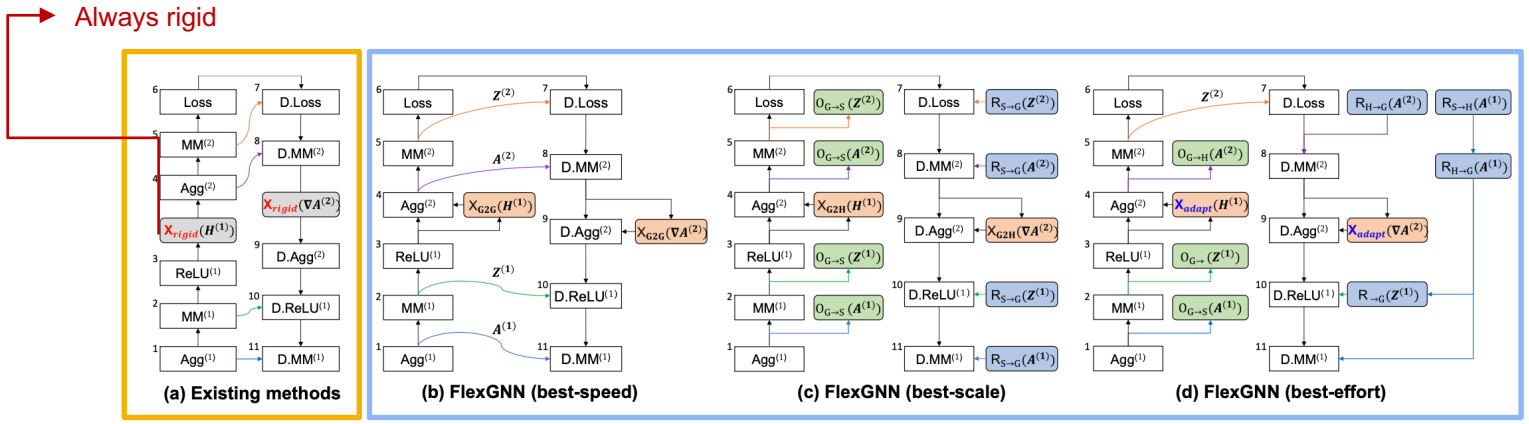
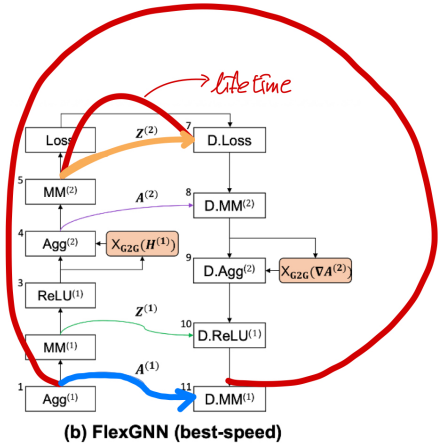


Figure 2: Comparison of training plans between existing methods and FlexGNN for a 2-layer GCN model of Figure 1.

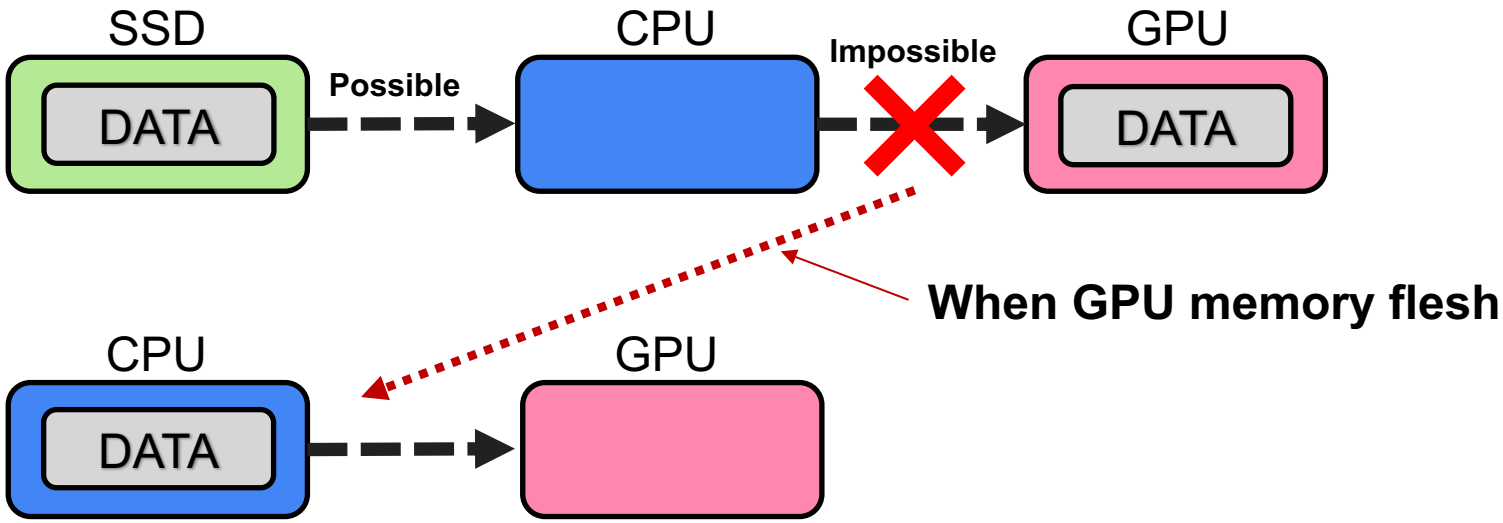
- **Lifetime of intermediate data**
  - Determine the optimal placement of these operators for each intermediate data, we will define its lifetime
  - Three steps in a plan: (1) being computed (2) being used in forward propagation, and (3) being reused in backward propagation
    - $pComp(d), pUse(d), pReUse(d)$

$$lifetime(d) = [pUse(d), pReUse(d)]$$



FlexGNN-Space of Training Plans

- **Space of Training Plans**
  - Scope rule: Offloading and reloading operators for intermediate data must be located within its lifetime.
  - Order rule: The reloading operator must come after the offloading operator for the same intermediate data.
  - Split rule: Intermediate data offloaded using $O_{G \to S}$ can be reloaded using two sequential but non-consecutive load operators



FlexGNN-Best Effort Approach

- **Exchange operator determination**
  - Determine the exchange operator before optimizing the operators for intermediate data
    - Since inter-GPU communication is the primary bottleneck in full-graph GNN training
    - $|XBuf_g| = 2\times|f_{max}|/g$
    - $|XBuf_h| = 2\times|f_{max}|$
    - $|f_{max}| = |V|\times \max\left(\dim_{out}^{(i)}\right), \begin{cases} 1 \le l < L, & for\ GCN \\ 1 \le l \le L, & for\ GAT \end{cases}$

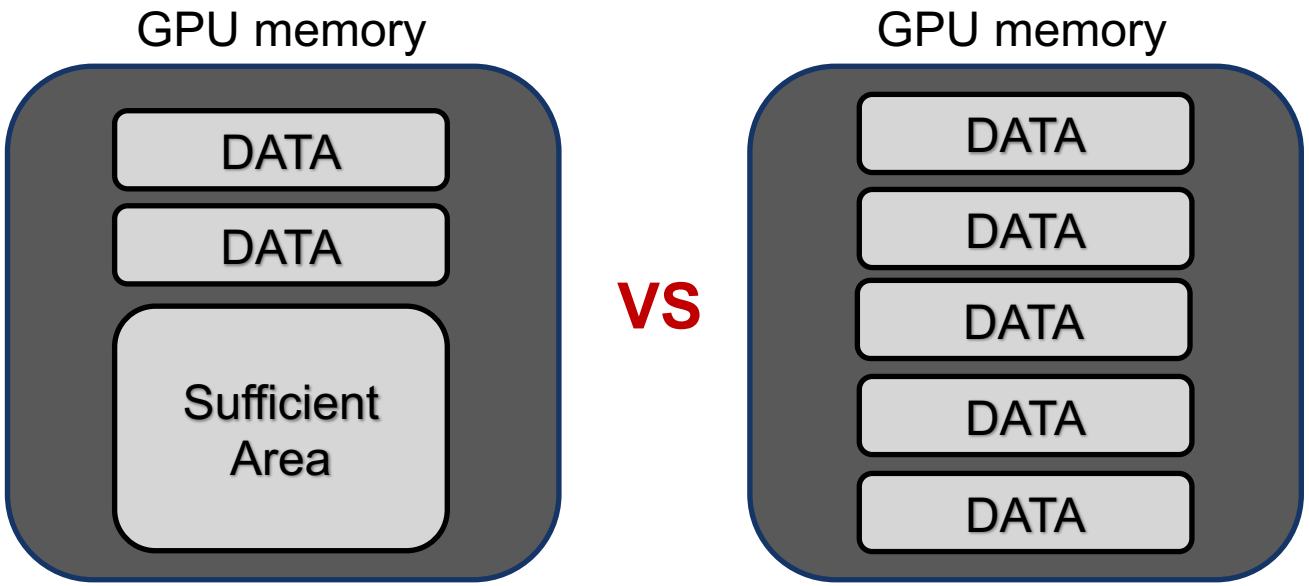
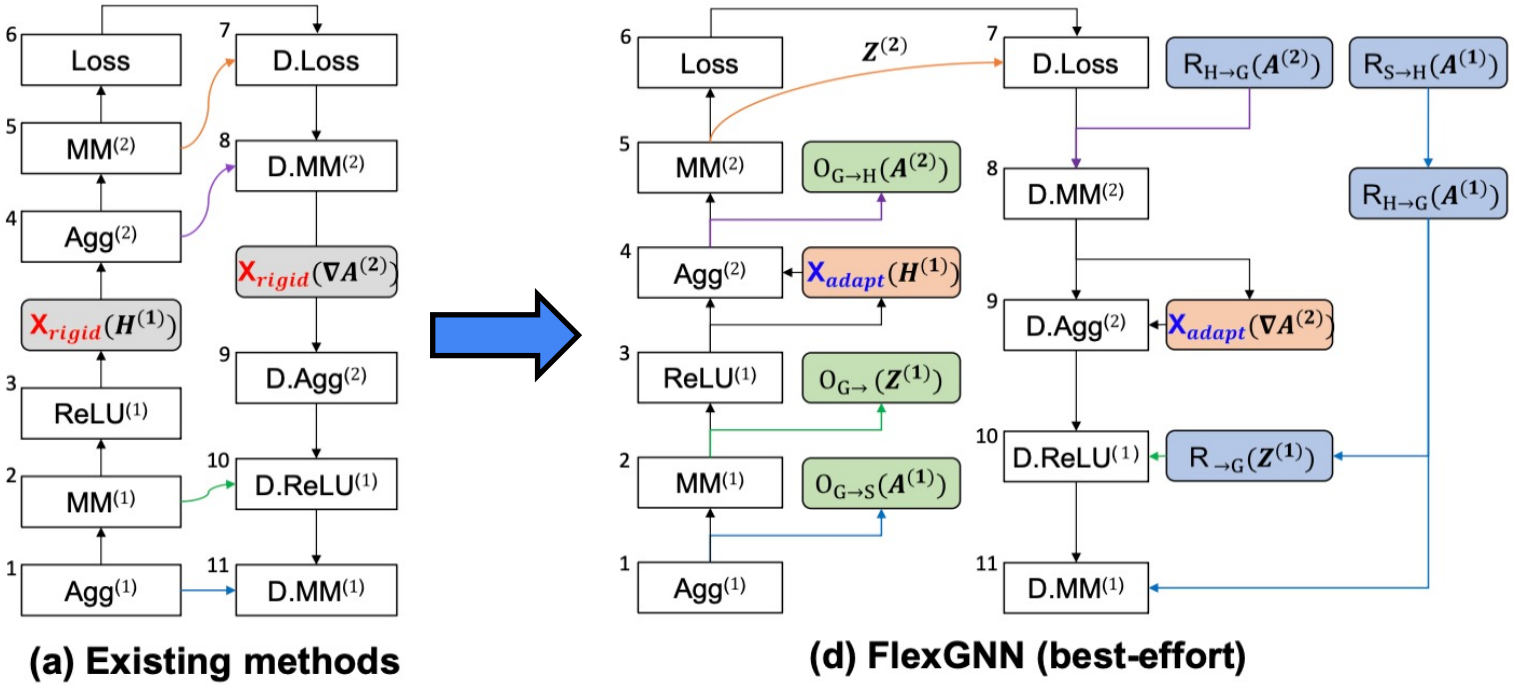


Table 4: Exchange operator determination (HW_g and HW_h are the hardware GPU and host memory sizes, respectively).

Oprator	available GPU memory ($\mathcal{M}_g$)	available host memory ($\mathcal{M}_h$)
X_{G2G}	$HW_g - \mu_g - \|XBuf_g\|$	$HW_h - \mu_h$
X_{G2H}	$HW_g - \mu_g$	$HW_h - \mu_h - \|XBuf_h\|$

FlexGNN-Best Effort Approach

- **Optimization goal**
  - Applying offloading and reloading operators to the basic plan, numerous plans can be generated



(a) Existing methods

(d) FlexGNN (best-effort)

$$\mathcal{P}^* = \underset{\mathcal{U}_g(\mathcal{P}) \le \mathcal{M}_g, \mathcal{U}_h(\mathcal{P}) \le \mathcal{M}_h}{\operatorname{argmin}} \left(C_{total}(\mathcal{P})\right)$$

$$C_{total}(\mathcal{P}) = \sum_{d_i \in D(\mathcal{P})} \left(\phi_i C_{move}(d_i) + \lambda_i C_{comp}(d_i)\right) - \sum_{\{d_i \in D(\mathcal{P}) \mid \phi_i = 1\}} \gamma_i(d_i) C_{move}(d_i)$$

FlexGNN-Intermediate Data Management Costs

- **Costs of data movement and recomputation**
  - There is no combination of $\phi_i = 1$ and $\lambda_i = 1$
  - Retention implies keeping the intermediate data in GPU memory without offloading

Table 5: Memory usage and time costs of each operator.

Variable		Operator	Memory usage		Time cost (i.e., $C_{move}(d)$ or $C_{comp}(d)$)
ϕ_i	λ_i		per GPU	host	
0	0	Retention	$\|d\|/g$	0	-
1	0	$O_{G\to H}$	0	$\|d\|$	$2\times\|d\|/B_{G\text{-}H}$
1	0	$O_{G\to S}$	0	0	$2\times(\|d\|/B_{G\text{-}H}+\|d\|/B_{H\text{-}S})$
0	1	$O_{G\to}$	0	0	$time(pComp(d))$

→ Using FLOPs & TFLOPs

→ $C_{comp}(d) = time(pComp(d)) = \frac{FLOPs(pComp(d))}{GPU\ TFLOPs}$

- **Hiding the cost of asynchronous operators**
  - based on data movement times and the estimated execution times

$$\gamma_i(d_i) = \min\left(0.5, \frac{[pOff(d_i), pLoss]}{C_{move}(d_i)/2}\right) + \min\left(0.5, \frac{[pRel(d_i), pReuse(d_i))}{C_{move}(d_i)/2}\right) \quad (6)$$

Offload Hiding　　Reload Hiding

FlexGNN-Optimization Algorithm

- **Target Selection**
  - Step to determine which operator (op) to apply to intermediate data d_i
  - Adopt Dynammic Programming

- **Operator Scheduling**
  - Optimizes the positions of offloading and reloading operations
  - Offloading and reloading operators are inserted into the training plan

Conclusions

- **Environments**
  - Neugraph
    - Multi-GPU server, which is equipped with dual 2.6 GHz
    - Intel Xeon E5-2690v4 processors (28 cores in total), 512 GB memory, and 8 NVIDIA Tesla P100 GPUs
  - ROC
    - Cluster: 4 nodes × (dual CPU + 256GB DRAM + 4 Tesla P100)
  - GNNAdvisor
    - 8-core 16-thread Intel Xeon Silver 4110 CPU and a Quadro P6000 GPU
    - Tesla V100 GPU on the DGX-1 system to demonstrate the generality of GNNAdvisor
  - FlexGNN
    - Four NVIDIA A100 (80 GB) GPUs
    - Two 16-core CPUs of 3.0 GHz, 512 GB Mem, 6 TB SSD

Conclusions

- **Overall**
  - A training approach that does not drop any data
  - Different papers propose various solutions to the three issues: partitioning, workload imbalance, and memory management
  - They address these problems with not only high-level ideas but also low-level techniques

- **Limitations**
  - Because many works assume multi-GPU or multi-node setups, they are difficult to reproduce and deploy